AI Hackathon
Route Optimization
2026

AI for Green Logistics

โครงการ AI Engineering สำหรับเยาวชนไทย

เชื่อมการเรียนรู้กับโจทย์ Route Optimization
จากอุตสาหกรรมจริง

- เวทีการแข่งขันด้าน AI Engineering
- ใช้ AI และ Data Science เพื่อแก้ปัญหา Route Optimization
- เรียนรู้จากอุตสาหกรรมจริง สู่ Green Logistics



AI Hackathon | Route Optimization 2026

BU + SCG-CPAC = AI Talent

SCHOOL OF ENGINEERING BANGKOK UNIVERSITY

Academic

+

SCG CPAC

Industry Partner

=



AI Talent

- BU ทำหน้าที่เชื่อมการเรียนรู้ด้านวิศวกรรม AI
- SCG-CPAC สะท้อนโจทย์และบริบทอุตสาหกรรมจริง
- ผู้เข้าแข่งขันได้ฝึกคิดและลงมือทำกับปัญหาที่ใกล้โลกจริง

โจทย์หลักที่โครงการมุ่งเน้น

01



Vehicle Routing & Network Optimization

เพิ่มประสิทธิภาพเส้นทาง
และเครือข่ายการขนส่ง

02



Operational Cost Reduction

ลดต้นทุนการดำเนินงาน
อย่างยั่งยืน

03



Energy Efficiency & Carbon Reduction

เพิ่มประสิทธิภาพพลังงาน
และลดการปล่อยคาร์บอน

รอบคัดเลือก: ทุกทีมต้องทำอะไรบ้าง?

4 ภารกิจหลัก เพื่อพิสูจน์ความพร้อมก่อนเข้าสู่รอบ Hackathon

QUALIFIER PACKAGE

01

Micro-Credential

OVRP Routing Game



02

CVRP Benchmark + Solver

A-n32-k5 / B-n31-k5 / P-n40-k5



03

Self-created Case

สร้างโจทย์ CVRP ที่เชื่อมกับ SCG-CPAC



04

Presentation

Slide + Video



ผลงานทั้งหมดจะถูกนำมาประเมินร่วมกัน **เพื่อคัดเลือกทีมเข้าสู่รอบ Hackathon**





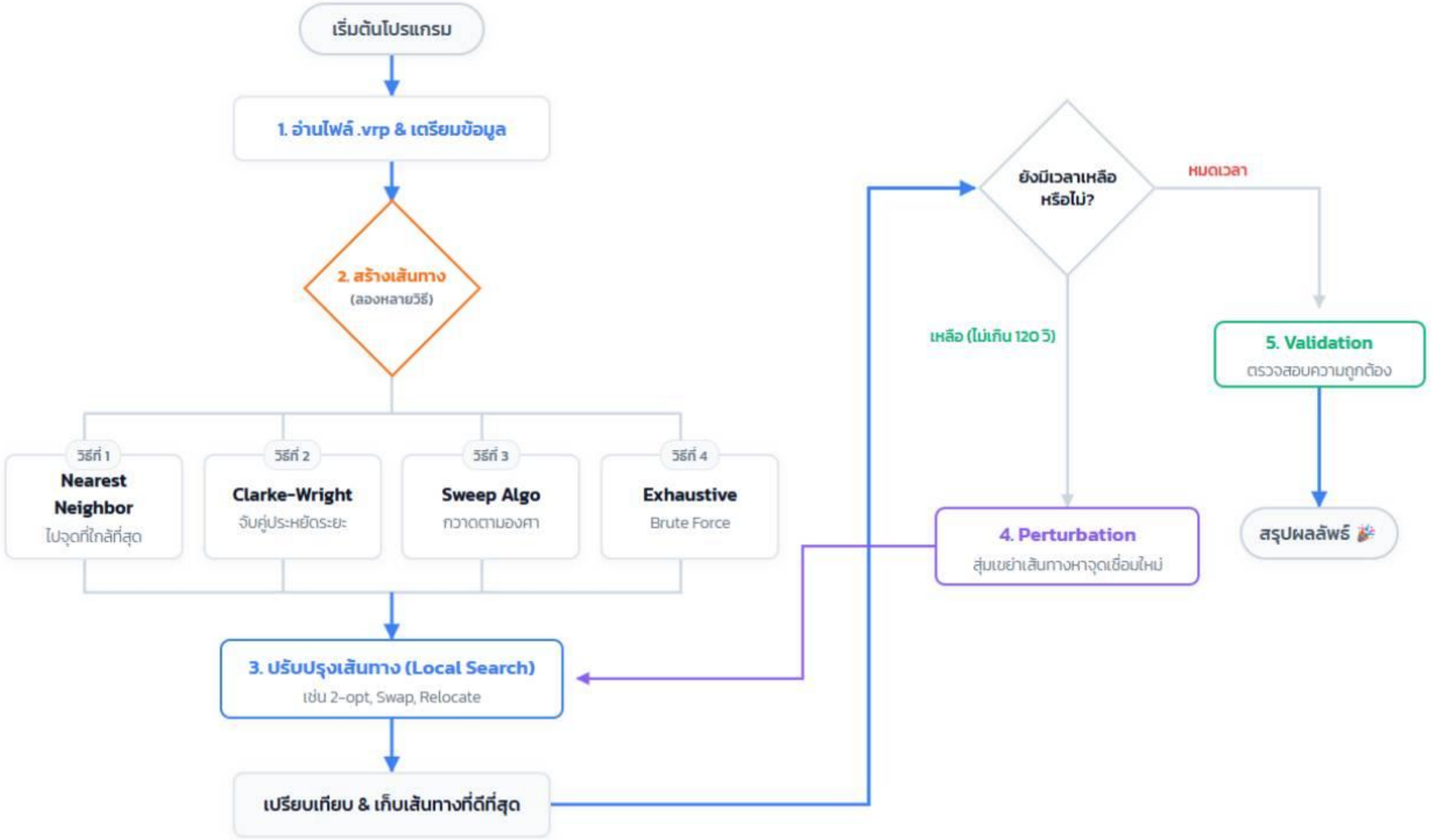
SOLVER





📌 โครงสร้างการทำงานของโปรแกรม CVRP

คลิกที่แต่ละขั้นตอน (กล่องข้อความ) ในแผนภาพเพื่อเรียนรู้รายละเอียดและอัลกอริทึมที่อยู่เบื้องหลัง







STAGE 1 :

INITIALIZATION AND DATA





Overview / ภาพรวม

📌 การเตรียมข้อมูลและตั้งค่าเริ่มต้น (Stage 1)

ก่อนที่โปรแกรมจะสามารถหาเส้นทางเดินรถได้ สิ่งแรกที่ต้องทำคือ **การอ่านข้อมูลโจทย์และเตรียมสภาพแวดล้อม** ให้พร้อมสำหรับการคำนวณ ข้อมูลเหล่านี้มักจะอยู่ในรูปแบบไฟล์ข้อความมาตรฐาน เช่น `.vrp`

> 💡 **ทำไมขั้นตอนนี้ถึงสำคัญ?**
> ข้อมูลที่ดีคือรากฐานที่สำคัญ (Garbage In, Garbage Out) หากอ่านพิกัดผิด หรือตั้งค่าความจุรถผิด อัลกอริทึมในขั้นตอนต่อๆ ไปก็จะได้ผลลัพธ์ที่ผิดเพี้ยนไปทั้งหมด

1. โหลดไฟล์และแยกแยะข้อมูล

โปรแกรมจะเปิดไฟล์ `.vrp` ขึ้นมาอ่านทีละบรรทัด ค้นหาคีย์เวิร์ดสำคัญ เช่น พิกัด (NODE_COORD_SECTION) และ ความต้องการสินค้า (DEMAND_SECTION)

2. สร้าง Distance Matrix

เพื่อความรวดเร็วในการทำงาน จะมีการคำนวณระยะทางระหว่างทุกจุดเตรียมเอาไว้ล่วงหน้า (Pre-calculation) เก็บไว้ในรูปแบบตาราง (Matrix)

โครงสร้างและวิธีการอ่านไฟล์ .vrp

📌 รูปแบบไฟล์มาตรฐาน

ไฟล์ .vrp มักจะใช้ฟอร์แมตของ TSPLIB ซึ่งแบ่งข้อมูลออกเป็นส่วนๆ (Sections) อย่างชัดเจน ดังนี้:

```
NAME : A-n32-k5
COMMENT : (Augerat et al, No of trucks: 5, Optimal value: 784)
TYPE : CVRP
DIMENSION : 32
EDGE_WEIGHT_TYPE : EUC_2D
CAPACITY : 100
NODE_COORD_SECTION
1 82 76 # รหัส 1 คือ Depot อยู่ที่พิกัด X=82, Y=76
2 96 44 # รหัส 2 คือ ลูกค้าคนที่ 1
DEMAND_SECTION
1 0 # Depot ไม่มีความต้องการสินค้า (Demand = 0)
2 19 # ลูกค้าคนที่ 1 ต้องการสินค้า 19 หน่วย
DEPOT_SECTION
1 # ระบุว่า Node ID 1 คือ Depot
-1 # เครื่องหมายจบรายการ Depot
EOF # สิ้นสุดไฟล์ (End of File)
```

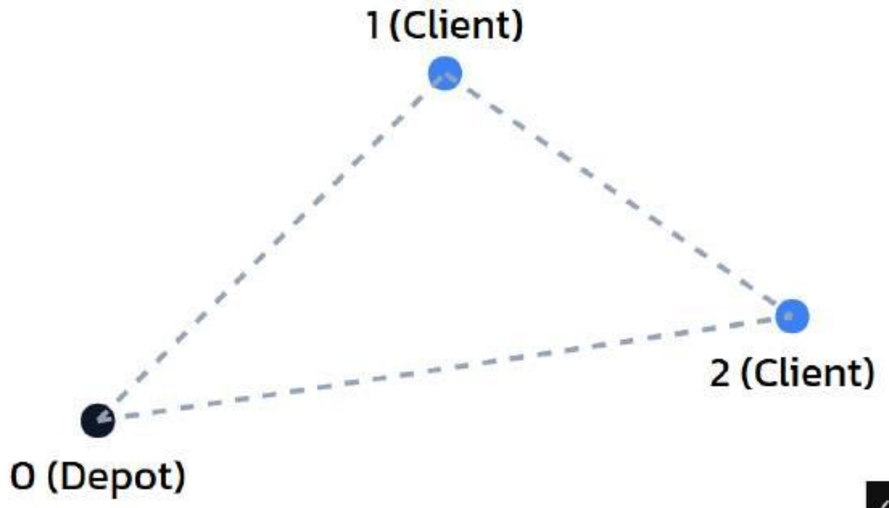
💻 การทำงานของโปรแกรม

1. **อ่าน Metadata:** เก็บค่า DIMENSION (จำนวนจุดทั้งหมดรวม Depot) และ CAPACITY (ความจุของรถแต่ละคัน)
2. **อ่าน Coordinates:** เมื่อเจอคำว่า NODE_COORD_SECTION โปรแกรมจะวนลูปอ่าน (ID, X, Y) มาเก็บไว้ใน Array หรือ List ของ Object
3. **อ่าน Demands:** เมื่อเจอคำว่า DEMAND_SECTION โปรแกรมจะวนลูปอ่าน (ID, Demand) และนำไปแมปกับลูกค้าที่บันทึกไว้ในขั้นตอนก่อนหน้า
4. **ระบุ Depot:** เมื่อเจอคำว่า DEPOT_SECTION โปรแกรมจะอ่านค่ารหัสที่ระบุให้เป็น Depot (มักจะเป็น 1) และเมื่อเจอค่า -1 หมายถึงจบรายการ Depot จากนั้นไฟล์จะจบด้วย EOF

การคำนวณ Distance Matrix

📌 ทำไมต้องทำ Distance Matrix ล่วงหน้า?

อัลกอริทึมเช่น Local Search จะมีการสลับจุดไปมานับล้านครั้ง หากต้องมานั่งคำนวณสูตร `sqrt((x2-x1)^2 + (y2-y1)^2)` ทุกครั้ง จะทำให้โปรแกรมช้ามาก เราจึงคำนวณเก็บไว้ในตาราง (2D Array) ล่วงหน้า



	0	1	2
0	0	14	21
1	14	0	12
2	21	12	0

$$\left(\sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}\right)$$

💻 โค้ดตัวอย่าง (Python)

```python
import math

def calculate_distance_matrix(nodes):
    n = len(nodes)
    matrix = [[0] * n for _ in range(n)]
    for i in range(n):
        for j in range(n):
            if i != j:
                dx = nodes[i].x - nodes[j].x
                dy = nodes[i].y - nodes[j].y
                dist = round(math.sqrt(dx**2 + dy**2)) # ปัดเศษตามมาตรฐาน TSPLIB
                matrix[i][j] = dist
    return matrix
```





STAGE 2 :

CONSTRUCTION HEURISTICS





📌 4 อัลกอริทึมการสร้างเส้นทางเริ่มต้น (Stage 2)

ในขั้นตอนการแก้ปัญหาการจัดเส้นทางเดินรถ (Vehicle Routing Problem - VRP) **"Stage 2"** คือขั้นตอนของการ **สร้างเส้นทางตั้งต้น (Construction Heuristics)** ซึ่งเปรียบเสมือนการร่างแบบแปลนการจัดส่งคร่าวๆ ก่อนที่จะนำไปขัดเกลาให้สมบูรณ์แบบใน Stage 3 (Local Search)

> 💡 **ทำไมต้องมีหลายอัลกอริทึม?**
> เนื่องจากแต่ละวิธีมีข้อดี-ข้อเสียต่างกัน โปรแกรมแก้ปัญหา (Solver) แบบ Multi-Strategy จึงไม่ได้เลือกใช้วิธีใดวิธีหนึ่ง แต่จะ **สั่งรันอัลกอริทึมทั้ง 4 ตัวนี้ควบคู่กันไป** เพื่อร่างแผนตั้งต้นหลายๆ แบบ จากนั้นจะเทียบดูว่าวิธีไหนให้ต้นทุนระยะทางต่ำที่สุด แล้วจึงหยิบแผนนั้นส่งต่อไปพัฒนาต่อ

1. Nearest Neighbor (NN)

เพื่อนบ้านที่ใกล้ที่สุด: เน้นความเร็ว ไปจุดที่ใกล้ที่สุดตรงหน้าเสมอ เหมาะกับงานเร่งด่วน

2. Clarke-Wright Savings

จับคู่ประหยัดระยะทาง: มาตรฐานอุตสาหกรรม คำนวณส่วนลดระยะทาง รวบเส้นทางที่คุ้มค่าที่สุดก่อน

3. Sweep Algorithm

กวาดตามรัศมีเข็มทิศ: แบ่งโซนลูกค้าตามองศา เส้นทางไม่ทับซ้อน แบ่งเขตรับผิดชอบชัดเจน

4. Exhaustive Search

แจกแจงทุกกรณี (Brute Force): ลองทุกรูปแบบเพื่อหาคำตอบที่ดีที่สุด 100% (ใช้ได้กับปัญหาสเกลจิ๋วเท่านั้น)

Algorithm 1 / 4

🚙 Nearest Neighbor (NN) – เพื่อนบ้านที่ใกล้ที่สุด

📌 ตรรกะ (Logic) & ขั้นตอน

ใช้วิธีคิดแบบ "โลภ" (Greedy) หรือการแก้ปัญหาเฉพาะหน้า คือ **"ตอนนี้อยู่ที่ไหน ให้ไปหาจุดที่ใกล้ที่สุดที่ยังไม่ได้ไปเสมอ"** โดยไม่สนว่าในอนาคตจะต้องอ้อมแค่ไหน

1. **เริ่มต้น:** รถบรรทุกเปล่าออกจากคลังสินค้า (Depot)
2. **ค้นหา:** ตรวจสอบลูกค้าที่ยังไม่ได้จัดส่งทั้งหมด ค้นหาคนที่ *ระยะทางใกล้ตำแหน่งปัจจุบันที่สุด* และ *ความต้องการสินค้ายังไม่เกินความจุที่เหลืออยู่ของรถ*
3. **เดินทาง:** รถเดินทางไปส่งของให้ลูกค้ารายนั้น นำของลงจากรถ (หักลบความจุ) และทำเครื่องหมายว่าส่งสำเร็จแล้ว
4. **ทำซ้ำ:** ทำขั้นตอนที่ 2-3 วนไปเรื่อยๆ จนกว่ารถจะเต็ม หรือส่งครบทุกคนแล้ว
5. **กลับคลัง:** รถวิ่งกลับคลังสินค้า หากยังมีลูกค้าเหลืออยู่ ให้เอารถคันใหม่เริ่มข้อ 1 อีกครั้ง

🚀 อัปเกรด: เทคนิค "NN seed" (Multi-start Nearest Neighbor)

ปัญหาของ NN ธรรมดา: โปรแกรมจะออกจาก Depot ไปหาจุดที่ใกล้ที่สุดเสมอ ทำให้ได้คำตอบเดิมแค่รูปแบบเดียว ซึ่งถ้าบังเอิญไปเจอ "ทางตัน" หรือต้องวิ่งอ้อมโลกในตอนจบ (Long tail) เราจะไม่มีทางเลือกอื่นเลย

การแก้ปัญหาด้วย NN seed: ในโค้ดการทำงานจริง โปรแกรมจะใช้เทคนิค **"บังคับจุดสตาร์ท (Seed)"** เข้ามาช่วย โดยแทนที่จะไปหาคนที่ใกล้สุดเสมอ โค้ดจะบังคับให้รถคันแรกวิ่งไปหาลูกค้ารายที่กำหนดเป็นจุดแรกสุดก่อน (ต่อให้จะอยู่ไกลแค่ไหนก็ตาม) จากนั้นค่อยกลับมาใช้กฎ NN ตามปกติ

ตัวอย่างการทำงาน:

- **NN ปกติ:** Depot → ใกล้สุด → ใกล้ถัดไป → ...
- **NN seed 1:** Depot → บังคับไปลูกค้า #1 → ใกล้ถัดไป → ...
- **NN seed 2:** Depot → บังคับไปลูกค้า #2 → ใกล้ถัดไป → ...

ผลลัพธ์เมื่อทำงานร่วมกับลูป:

โค้ดจะรันลูป **NN seed ตั้งแต่คนที่ 1 ถึง N (ครบทุกคน)** ทำให้ได้แผนที่ร่างตั้งต้นหน้าตาไม่ซ้ำกันหลายสิบแบบ! เมื่อส่งทั้งหมดให้ **Stage 3 (LS)** ขัดเกลาต่อ จึงง่ายที่จะดัดจนเจอเส้นทางที่สมบูรณ์แบบที่สุด (0.00% Gap)

🎯 เหมาะกับปัญหาแบบไหน?

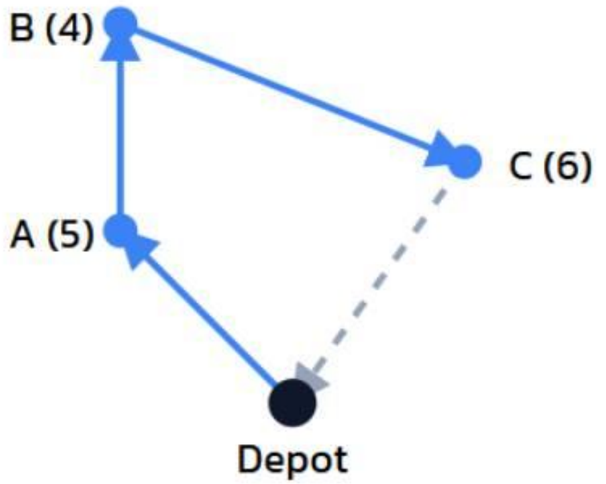
เหมาะกับปัญหาที่มี **จุดจัดส่งกระจุกตัวกันหนาแน่น (Clustered)** ปัญหาที่ต้องการคำตอบแบบเร่งด่วน (Real-time routing) หรือระบบที่มีทรัพยากรจำกัดมากๆ เพราะไม่ต้องใช้เวลาคำนวณซับซ้อน

ผลลัพธ์ที่ได้:

ได้เส้นทางที่ดูสมเหตุสมผลในช่วงแรก แต่ช่วงท้ายของเส้นทางมักจะย่ำแย่ (เกิด Long tail) เพราะรถต้องวิ่งไปเก็บตกลูกค้าที่อยู่ไกลลิบในตอนจบ ผลลัพธ์โดยรวมจึงมีความยาวระยะทางมากกว่าวิธีอื่นๆ เล็กน้อย แต่แลกมาด้วยความเร็ว

💻 ลำดับการทำงานของโค้ดจริง & ตัวอย่าง

โจทย์: Depot มีพัสดุส่ง A, B, C (รถ 1 คันจุพอทุกคน)
ระยะห่าง: Depot→A (5), Depot→C (10), A→B (4), B→C (6)



1. ออกจาก Depot ไป **A** (ใกล้สุด 5 กม.)
2. จาก A ไป **B** (ใกล้สุด 4 กม.)
3. จาก B ไป **C** (เหลือที่เดียว 6 กม.)
4. จาก C กลับ **Depot** (10 กม.)

รวมระยะทาง: 25 กม.

Algorithm 2 / 4

✂️ Clarke-Wright Savings – จับคู่ประหยัดระยะทาง

📌 ตรรกะ (Logic) & ขั้นตอน

เริ่มต้นจาก "สถานการณ์ที่แย่ที่สุด" คือ ใช้รถ 1 คันต่อลูกค้า 1 คน (วิ่งไปแล้วกลับเลย) จากนั้นคำนวณหา **"ส่วนลดระยะทาง (Savings)"** หากนำลูกค้า 2 คนมารวมอยู่ในรถคันเดียวกัน แล้วเลือกจับคู่ลูกค้าที่ลดระยะทางได้มากที่สุดก่อน

1. **สร้าง Baseline (วิ่งแยกคัน):** สมมติให้ลูกค้าทุกคนมีรถไปส่งแบบแยกคัน `(Depot -> ลูกค้า -> Depot)`
2. **คำนวณค่า Savings:** สำหรับลูกค้าทุกๆ คู่ (เช่น A กับ B) คำนวณว่าถ้ารวม A กับ B จะประหยัดทางได้เท่าไหร่
   `Savings(A,B) = ระยะทาง(Depot, A) + ระยะทาง(Depot, B) - ระยะทาง(A, B)`
3. **จัดเรียง:** นำค่า Savings ทุกคู่มาเรียงลำดับจาก "ประหยัดมากสุด" ไป "ประหยัดน้อยสุด"
4. **จับคู่ (Merge):** เริ่มพิจารณาจากคู่ที่ประหยัดสุด ถ้ารถยังมีความจุเหลือพอ และลูกค้าทั้งสองคนนี้ยังอยู่ที่ "ปลายสาย" ของเส้นทางปัจจุบัน ให้รวบเส้นทางเข้าด้วยกัน
5. **ทำซ้ำ:** จนกว่าจะตรวจสอบครบทุกคู่

🎯 เหมาะกับปัญหาแบบไหน?

เหมาะกับปัญหาโลจิสติกส์การขนส่งสินค้าขนาดใหญ่ที่มี **"ข้อจำกัดด้านความจุของรถ (Capacity)"** เป็นอุปสรรคสำคัญ และเหมาะกับธุรกิจที่มีคลังสินค้าเป็นศูนย์กลาง (Hub) คอยกระจายสินค้าออกไปยังพื้นที่รอบนอก

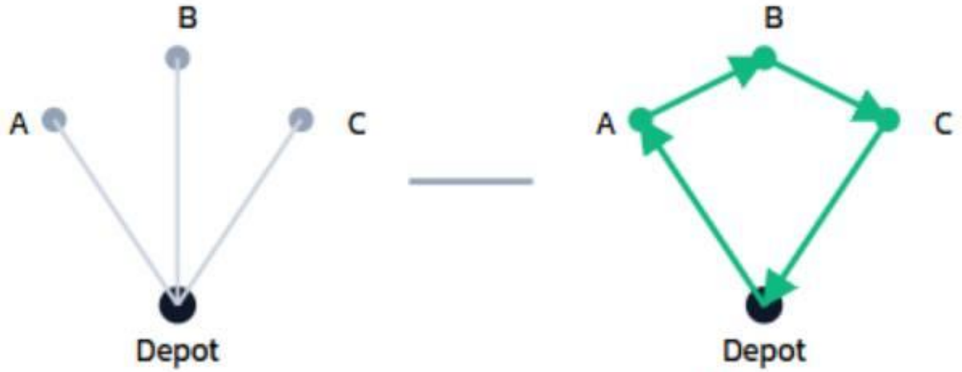
ผลลัพธ์ที่ได้:

ผลลัพธ์ที่ได้มีคุณภาพสูงมากและใกล้เคียงระดับดีที่สุด (Near-optimal) โดยรูปทรงของเส้นทางมักจะออกมาเป็นรูปร่างคล้ายกลีบดอกไม้ (Petal-shaped) คือรถแต่ละคันจะวิ่งออกไปเป็นวงกลมย่อยๆ แล้วกลับมาที่คลังอย่างเป็นระเบียบ

💻 ลำดับการทำงานของโค้ดจริง (Iterative Merge)

โจทย์: A=10กม., B=12กม., C=15กม. (รถจุ 3 คน)

Savings: คู่ B,C (17) > คู่ A,B (14) > คู่ A,C (5)



1. โค้ดเห็น **คู่ B,C ประหยัดสุด (17)** → ตัดเส้นวิ่งแยก แล้วสร้างรถคันใหม่วิ่ง D → B → C → D
2. โค้ดเห็น **คู่ A,B ประหยัดรองลงมา (14)** → B เป็นปลายสายเชื่อม Depot อยู่ โค้ดจึงปลดล็อค B แล้วเอา A มาต่อกลายเป็น D → A → B → C → D

Algorithm 3 / 4

🧭 Sweep Algorithm – กวาดตามรัศมี (เข็มทิศ)

📌 ตรรกะ (Logic) & ขั้นตอน

จัดกลุ่มลูกค้าตาม "ทิศทางและโซนที่ตั้ง" โดยอาศัยพิกัดเชิงมุม (Polar Coordinates) เหมือนมีเรดาร์กวาดรัศมีจากคลังสินค้าออกไปรอบๆ ลูกค้าที่อยู่ทิศทางเดียวกันจะถูกจัดขึ้นรถคันเดียวกัน

1. **หาพิกัดมุม:** ลากเส้นสมมติจาก Depot (0,0) ไปหาลูกค้าแต่ละคน คำนวณว่าลูกค้านั้นทำมุมกี่องศา (0 ถึง 360) กับ Depot
2. **เรียงลำดับองศา:** เรียงลูกค้าตามองศา เช่น จากน้อยไปมาก (กวาดทวนเข็มนาฬิกา)
3. **จัดคนขึ้นรถ (Sweep):** เริ่มกวาดเรดาร์ เอาลูกค้าเรียงใส่รถคันแรกไปเรื่อยๆ
4. **เปลี่ยนคัน:** เมื่อรถรับคนเพิ่มแล้ว "ความจุล้น" ให้ปิดเส้นทางรถคันนั้น แล้วเริ่มคันใหม่รับลูกค้าคนต่อไปทันที
5. **Multi-start (ออปชันเสริม):** หมุนจุดเริ่มต้น (เช่น เปลี่ยนไปเริ่มที่ 10, 20 องศา) เพื่อหาผลลัพธ์ที่ดีที่สุด

🎯 เหมาะกับปัญหาแบบไหน?

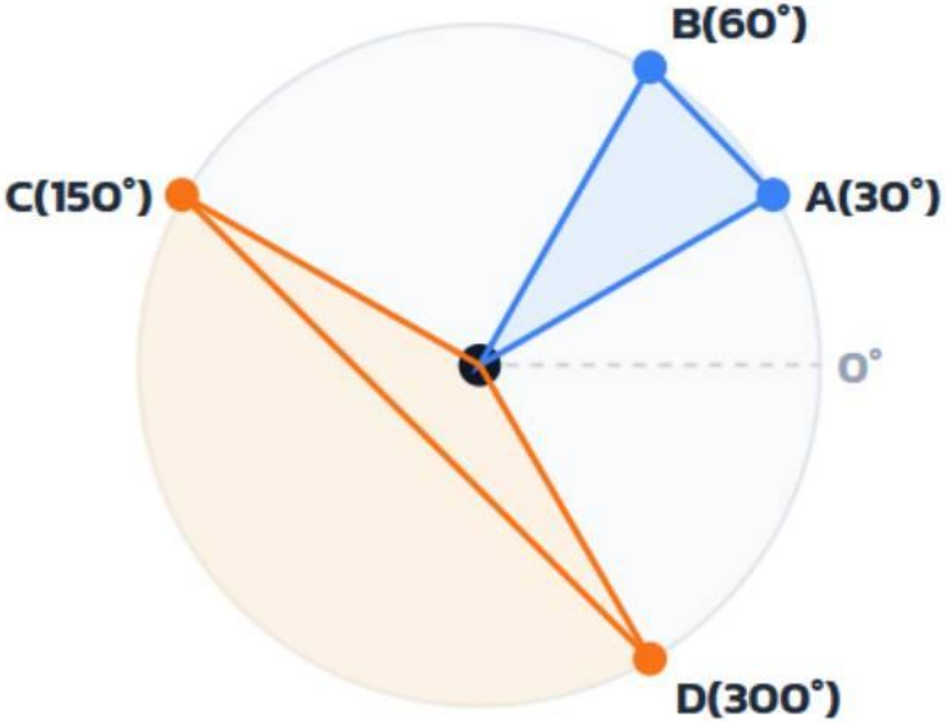
เหมาะกับปัญหาที่ลูกค้ากระจายตัวอยู่รอบคลังสินค้าในทุกทิศทาง หรือธุรกิจที่ **ต้องการแบ่ง "โซนรับผิดชอบ" ให้คนขับรถแต่ละคนอย่างชัดเจน** (เช่น ไปรษณีย์ หรือพนักงานส่งพัสดุที่คุ้นเคยกับซอยในพื้นที่ของตัวเอง)

ผลลัพธ์ที่ได้:

ได้เส้นทางที่ถูกแบ่งออกเป็นภูมิภาค (Zoning) อย่างชัดเจน เส้นทางของรถแต่ละคันแทบจะไม่ตัดข้ามหรือทับซ้อนกันเลย (No cross-over routes) แต่มักต้องใช้ Local Search มาช่วยดัดเส้นทางภายในโซนให้สั้นลงอีกที

💻 ลำดับการทำงานของโค้ดจริง (Zoning)

โจทย์: รถจุได้ 2 คน, มีลูกค้า A(30°), B(60°), C(150°), D(300°)



1. เริ่มกวาดที่ 0° ทวนเข็ม เจอ **A(30°)** ใส่ **รถคันที่ 1**
2. เจอ **B(60°)** ใส่รถคันที่ 1 (ความจุเต็ม 2/2) → ปิดรถคันที่ 1
3. กวาดต่อเจอ **C(150°)** ใส่ **รถคันที่ 2**
4. กวาดเจอ **D(300°)** ใส่รถคันที่ 2 (ความจุเต็ม) → ปิดรถคันที่ 2

Algorithm 4 / 4

🧮 Exhaustive Search – แจกแจงทุกกรณี (Brute Force)

📌 ตรรกะ (Logic) & ขั้นตอน

"ความพยายามอยู่ที่ไหน ความสำเร็จอยู่ที่นั่น" วิธีนี้ไม่ใช้ตรรกะการคาดเดาใดๆ แต่ใช้พลังประมวลผลของคอมพิวเตอร์ลองสร้าง **"ความเป็นไปได้ทั้งหมด"** ออกมาคำนวณระยะทางตรงๆ

💻 **ลำดับการทำงานของโค้ดจริง (Code Execution Sequence):**

1. **Generate Permutations:** โค้ดใช้ฟังก์ชัน (เช่น `itertools.permutations`) สร้าง Array รูปแบบการเรียงลำดับลูกค้าที่เป็นไปได้ทั้งหมด
2. **Route Partitioning & Validation:** นำแต่ละรูปแบบมาจำลองตัดแบ่งขึ้นรถบรรทุก หากรวมกันแล้วไม่เกินความจุรถ ก็ถือว่าเป็น "เส้นทางที่ถูกกฎ (Valid Route)"
3. **Cost Evaluation:** นำ Valid Route มาคำนวณระยะทางรวม (Cost) แบบตรงไปตรงมา
4. **Find Minimum:** โค้ดจะวนลูปอัปเดตเก็บค่า "ระยะทางที่น้อยที่สุด (Best Cost)" เอาไว้ เมื่อวนลูปครบ ก็จะคืนค่าเส้นทางที่ Cost ต่ำสุดทันที

🎯 เหมาะกับปัญหาแบบไหน?

เหมาะกับปัญหาที่มี **สเกลขนาดเล็กมากๆ เท่านั้น (Micro-scale)** เช่น จำนวนลูกค้าน้อยกว่า 10 ราย หรือถูกประยุกต์ใช้เป็นเพียง "เครื่องมือย่อย" ในการจัดเรียงเส้นทางภายในกลุ่มลูกค้าเล็กๆ (Sub-tour) ที่ถูกตัดแบ่งมาแล้ว

ผลลัพธ์ที่ได้:

ได้ "เส้นทางที่สั้นที่สุดแบบ 100% (Global Optimum)" รับประกันว่าไม่มีเส้นทางไหนดีไปกว่านี้อีกแล้ว แต่มักจะต้องแลกมาด้วยเวลาประมวลผลที่มหาศาล (เติบโตแบบ Factorial $O(N!)$)

ตัวอย่างการ Generate Permutations (ลูกค้า 3 คน)

มีลูกค้า A, B, C (รวม 3! = 3x2x1 = 6 รูปแบบ)

```
# คอมพิวเตอร์สร้างรูปแบบทั้งหมดขึ้นมา:
1. D → A → B → C → D -- Cost = 40 (✨ MIN)
2. D → A → C → B → D -- Cost = 55
3. D → B → A → C → D -- Cost = 48
4. D → B → C → A → D -- Cost = 60
5. D → C → A → B → D -- Cost = 60
6. D → C → B → A → D -- Cost = 40 (✨ MIN)

return Best_Route # คืนค่าเส้นทางที่ 1 ทันที
```

หมายเหตุ: เส้นทางที่ 1 และ 6 มีระยะทางเท่ากัน เพราะเป็นการวิ่งย้อนศร (Reverse) ของเส้นทางเดียวกัน





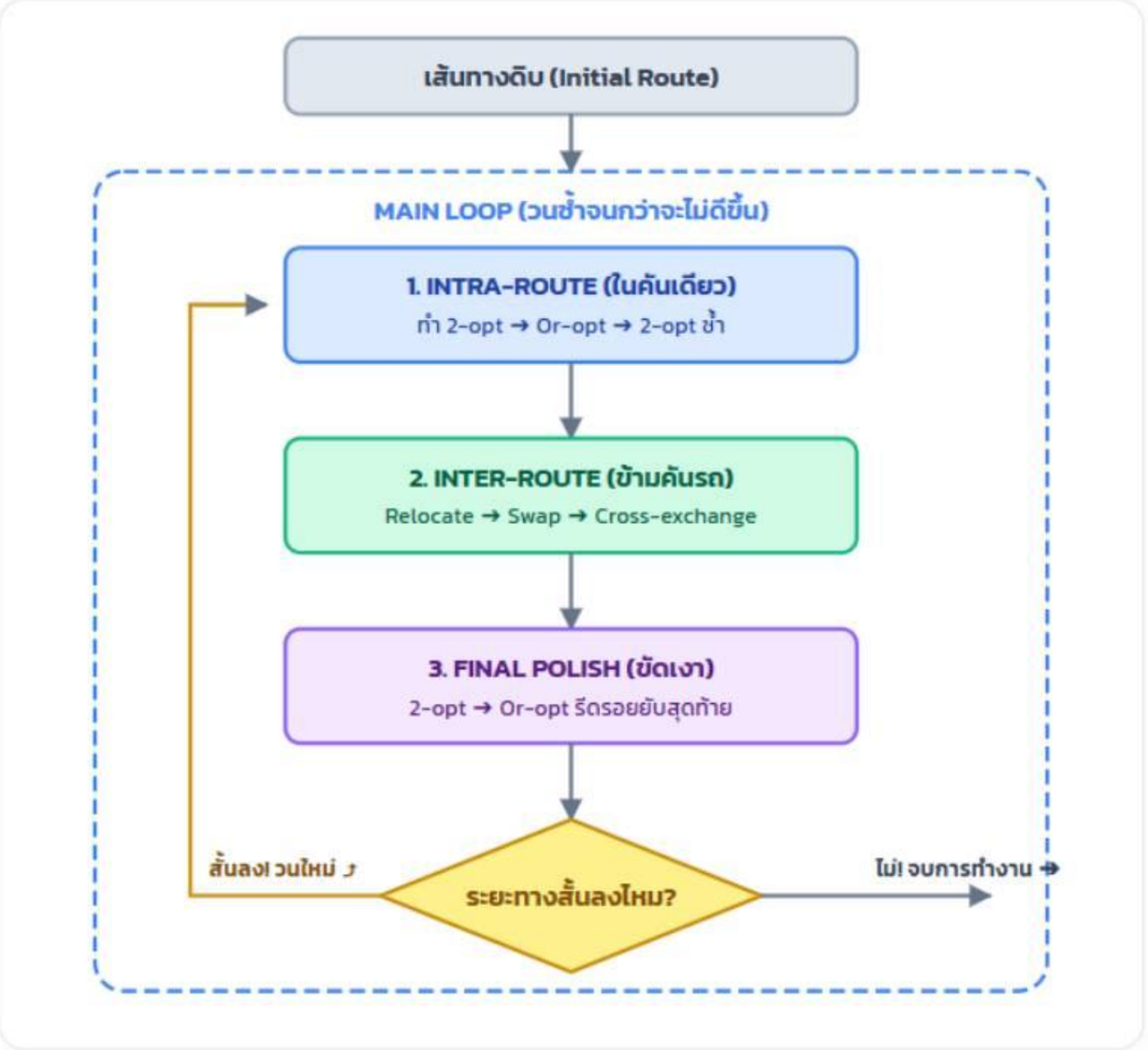
STAGE 3 :

LOCAL SEARCH





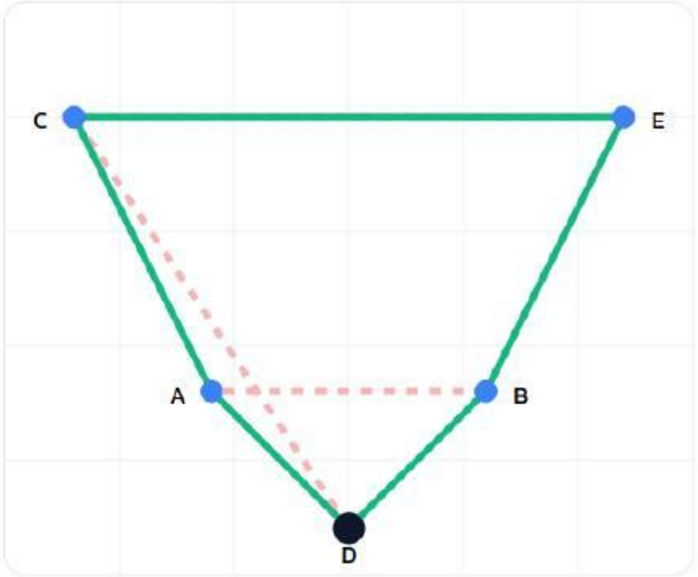
ผังการทำงาน (Flowchart)



INTRA-ROUTE

🔄 2-opt (แก้เส้นกากบาท)

Depot(5,0), A(3,2), B(7,2), C(1,6), E(9,6)



- **เส้นไขว้ก่อน:** D → A → [B → E → C] → D (รวม 26)
- **หลัง 2-opt:** D → A → [C → E → B] → D (รวม 22)
- **ผลลัพธ์:** สลับ Array ส่วน [B,E,C] กลับด้าน เส้นทางหายกากบาท! ประหยัดไป 15.4%

💻 โค้ดคอมพิวเตอร์ไล่ตรวจสอบ 2-opt อย่างไร? (Step-by-Step Trace)

ในความเป็นจริง โค้ดไม่มีดวงตามองแผนที่ แต่มันใช้ **Nested Loops *i* และ *j*** กวาดหาจุดไขว้จาก Array `[D, A, B, E, C, D]` แบบถึกทน (Brute-force) ดังนี้:

รอบที่ 1: กำหนดจุดตัดแรก *i* อยู่หลัง A (Index 1) เลื่อนจุดตัดสอง *j* ไปเรื่อยๆ

- *j* อยู่หลัง **B**: เทียบเส้นเดิม (D–A, B–E = 3+4=7) กับเส้นไขว้ (D–B, A–E = 3+7=10) → 10 < 7? ❌ **สลับแล้วแย่ลง**
- *j* อยู่หลัง **E**: เทียบเส้นเดิม (D–A, E–C = 3+8=11) กับเส้นไขว้ (D–E, A–C = 7+4=11) → 11 < 11? ❌ **ไม่ดีขึ้น**
- *j* อยู่หลัง **C**: เทียบเส้นเดิม (D–A, C–D = 3+7=10) กับเส้นไขว้ (D–C, A–D = 7+3=10) → 10 < 10? ❌ **ไม่ดีขึ้น**

รอบที่ 2: เลื่อนจุดตัดแรก *i* มาอยู่หลัง B (Index 2)

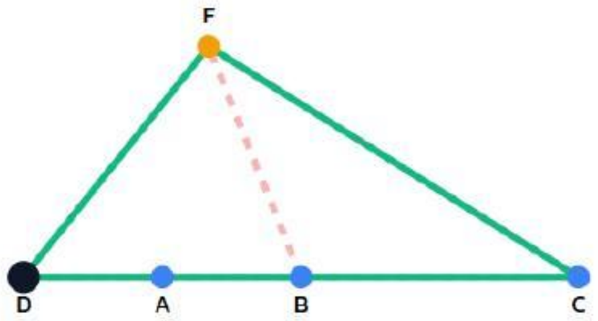
- *j* อยู่หลัง **E**: เทียบเส้นเดิม (A–B, E–C = 4+8=12) กับเส้นไขว้ (A–E, B–C = 7+7=14) → 14 < 12? ❌ **สลับแล้วแย่ลง**
- *j* อยู่หลัง **C**: เทียบเส้นเดิม (A–B, C–D = 4+7=**11**) กับเส้นไขว้ (A–C, B–D = 4+3=**7**) → **7 < 11?** ✅ **คุ้มมาก!**

นี่คือวินาทีที่อัลกอริทึม **"จับได้ (First Accept)"** ว่าสลับตรงนี้แล้วระยะทางลดลง 4 กิโลเมตร! มันจึงหลุดเข้าไปทำคำสั่งพลิก Array ทันที

INTRA-ROUTE

✂️ Or-opt (แก้คนหลงคิว)

Depot(0,5), A(3,5), B(6,5), C(12,5), F(4,10)



- **ก่อนดึง:** D → A → B → [F] → C → D (รวม 32)
- **หลังดึง:** D → A → B → C → [F] → D (รวม 27)
- **ผลลัพธ์:** ดึง F ออกมาจากหลัง B แล้วไปเสียบแทรกหลัง C ประหยัดไป 15.6%

💻 โค้ดคอมพิวเตอร์ทำงาน Or-opt อย่างไร? (Step-by-Step Trace)

อัลกอริทึมจะ **"ดึง F ออกมาจากตำแหน่งเดิม"** แล้วลองคำนวณการ **"เสียบเข้าไปทุกรอยต่อ"** ของเส้นทางที่เหลือ ดูว่าตรงไหนทำให้ระยะทางรวมสั้นที่สุด:

📌 ขั้นตอนที่ 1: ดึง F ออกจากเส้นทาง

เหลือเส้นทางหลัก: `D → A → B → C → D`

📌 ขั้นตอนที่ 2: ลองเสียบ F เข้าไปทุกช่องว่าง

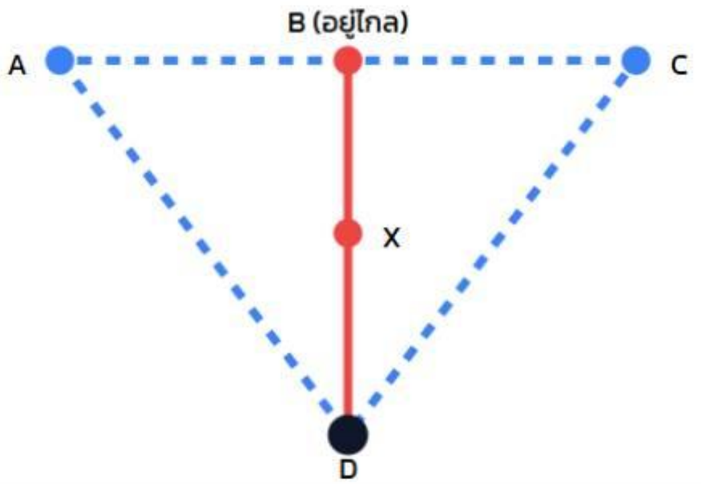
ตำแหน่งที่แทรก F	จำลองเส้นทางใหม่	ระยะทางรวม
ระหว่าง **D** กับ **A**	D → **F** → A → B → C → D	6+5+3+6+12 = 32 (แย่กว่าเดิม)
ระหว่าง **A** กับ **B**	D → A → **F** → B → C → D	3+5+5+6+12 = 31 (ดีขึ้นนิดหน่อย)
ระหว่าง **B** กับ **C**	D → A → B → **F** → C → D	3+3+5+9+12 = 32 (ที่อยู่เดิม)
ระหว่าง **C** กับ **D**	D → A → B → C → **F** → D	**3+3+6+9+6 = 27** ✅ 🏆 **ดีที่สุด!**

หัวใจสำคัญของการประหยัด: ระยะกลับบ้านเดิมคือ **C → D (12)** ซึ่งไกลมาก! การย้าย F(4,10) ที่อยู่ใกล้คลังมากกว่า ไปวางไว้ปิดท้ายเป็น **F → D (6)** จึงช่วยหั่นระยะทางขากลับลงได้ถึงครึ่งนึงทันที

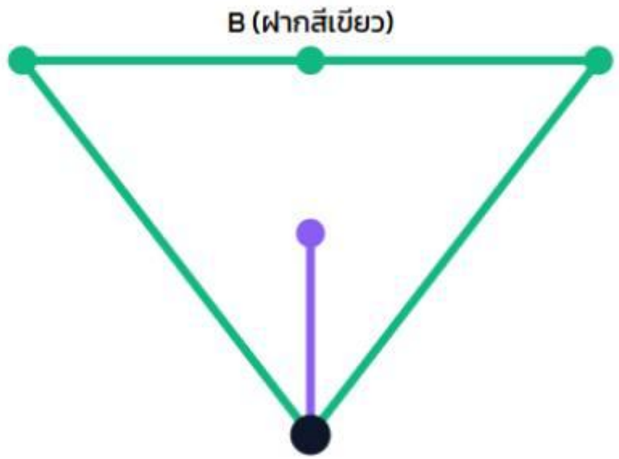
INTER-ROUTE

📦 1. Relocate (ย้ายลูกค้าข้ามคัน)

เหมือนบุรุษไปรษณีย์คนที่ 2 โยนของให้คนที่ 1 เพราะคนที่ 1 ขับรถผ่านอยู่แล้ว



ก่อน: รถคันสีแดงต้องขับยาวไปรับ B



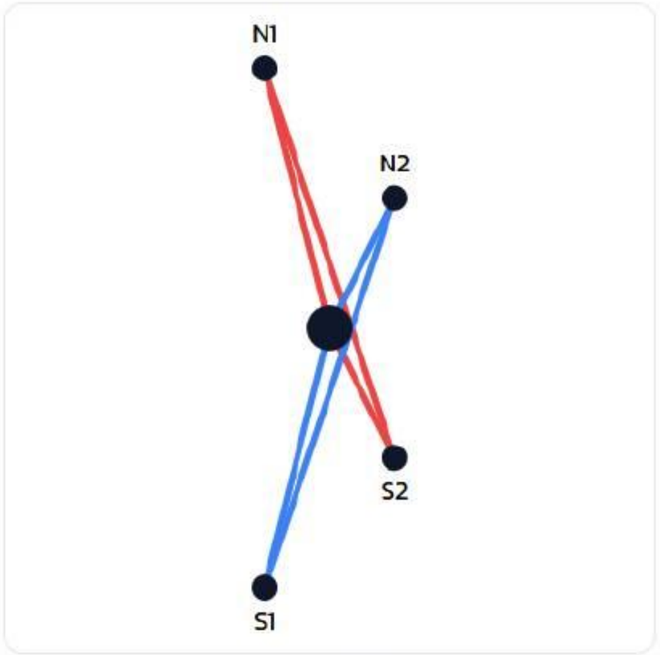
หลัง: โยน B ให้คันสีเขียววิ่งรับแทน (ประหยัดระยะ 18%)

- **ก่อนย้าย:** รถ 1: คลัง → A → C → คลัง | รถ 2: คลัง → X → [B] → คลัง
- **หลังย้าย:** รถ 1: คลัง → A → [B] → C → คลัง | รถ 2: คลัง → X → คลัง
- **ผลลัพธ์:** ดึง B ออกจากรถ 2 ไปเสียบแทรกในเส้นทางรถ 1 ที่วิ่งผ่านอยู่แล้ว ทำให้รถ 2 ไม่ต้องขับอ้อมไกล

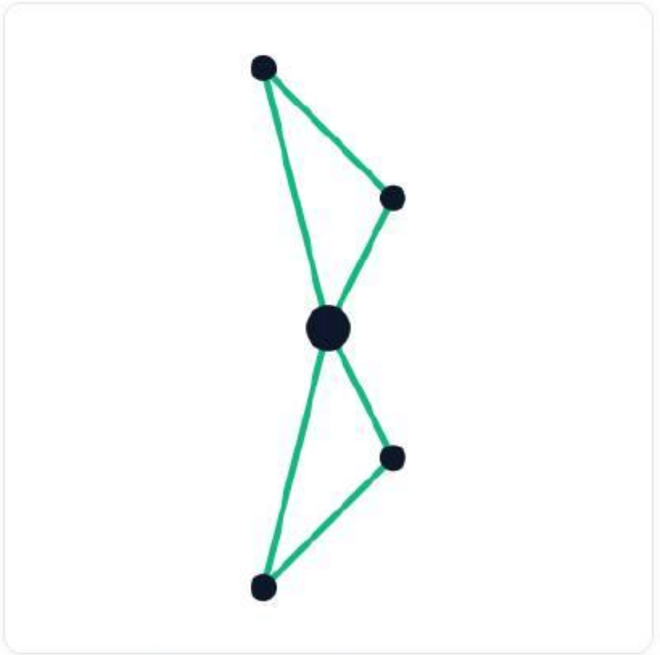
INTER-ROUTE

🔀 2. Swap (แลกตัว 1-1)

เหมือน Messenger 2 คนถือของผิดโซนกัน เลยนัดเจอแลกของกัน



ก่อน: ต่างคนต่างวิ่งข้ามโซนไปหา N2 กับ S2



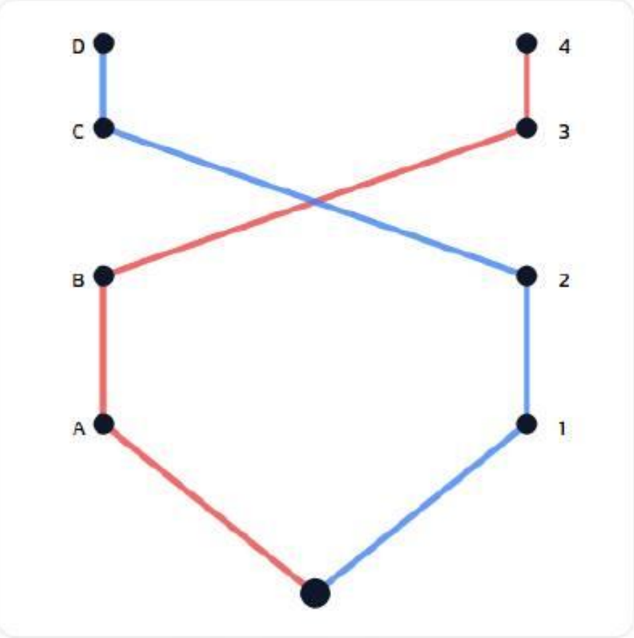
หลัง: แลกตัวกัน แบ่งโซนชัดเจน (ประหยัด 33%)

- **ก่อนสลับ:** รถ 1: คลัง → N1 → [S2] → คลัง | รถ 2: คลัง → S1 → [N2] → คลัง
- **หลังสลับ:** รถ 1: คลัง → N1 → [N2] → คลัง | รถ 2: คลัง → S1 → [S2] → คลัง
- **ผลลัพธ์:** แลกตัว S2 กับ N2 คืนให้ถูกโซน (เหนือ/ใต้) ทำให้เส้นทางวิ่งเป็นวงสวยงาม ไม่กากบาทข้ามโซนกัน

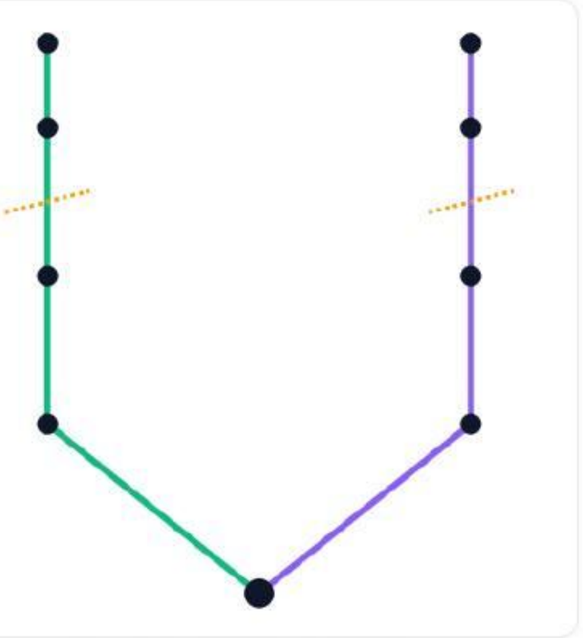
INTER-ROUTE

🔄 3. Cross-exchange (สลับเป็นขบวน / สลับสาย)

เหมือนรถ 2 คันวิ่งขนานกันมาดีๆ แต่ครึ่งหลังดันหักเลี้ยวไขว้กันเป็นรูปตัว X



ก่อน: เส้นทางตัดกันเป็นกากบาทกลางอากาศ



หลัง: ตัดแล้วไขว้ก่อนขนสลับกัน วิ่งเป็นเส้นขนาน!

- **ก่อนสลับขบวน:** รถ 1: คลัง → A → B → [3 → 4] → คลัง | รถ 2: คลัง → 1 → 2 → [C → D] → คลัง
- **หลังสลับขบวน:** รถ 1: คลัง → A → B → [C → D] → คลัง | รถ 2: คลัง → 1 → 2 → [3 → 4] → คลัง
- **ผลลัพธ์:** เอากรรไกรตัดรอยต่อตรงจุดที่ไขว้กันเป็นรูปตัว X แล้วสลับขบวนท่อนปลายกัน ทำให้เส้นทางขนานกันพอดี!





STAGE 4 :

PERTURBATION






คำตอบที่ดีที่สุดปัจจุบัน
(Local Optimum)
วนลูปครบ 50 รอบ
หรือหมดเวลา 120 วิ?
ยังไม่จบ
สุ่มเขย่าเส้นทาง (Perturbation)
วิธีที่ 1
วิธีที่ 2
วิธีที่ 3
Reverse Segment
สลับทิศทางในกลุ่ม
Move Customer
ย้ายลูกค้าข้ามรถ
Swap Between
จับสลับลูกค้าคนละคัน
ส่งไป Local Search
จัดระเบียบให้เรียบร้อย
ระยะทางรวม
สั้นกว่าเดิมไหม?
สั้นกว่า!
บันทึกเป็น
คำตอบที่ดีที่สุดใหม่
ยาวกว่า
โยนทิ้ง
เริ่มใหม่จากตัวเดิม
ครบ 50 รอบ / หมดเวลา
จบ – ได้ผลลัพธ์ที่ปรับปรุงแล้ว

🎲 3 วิธีการสุ่มแบบต่างๆ

ในฟังก์ชัน `perturb_routes` โปรแกรมจะใช้เครื่องมือสุ่มของ Python ที่ชื่อว่า `random.choice()` เพื่อจับฉลากเลือกชื่อวิธีทำลายเส้นทาง 1 วิธี จากลิสต์ 3 วิธี แล้วนำชื่อนั้นมาเก็บในตัวแปรที่ชื่อว่า `method`

```
# จับฉลากสุ่ม 1 ใน 3 วิธี
method = random.choice(["reverse_segment", "move_customer", "swap_between"])
```

🔀 1. สลับทิศทางแบบสุ่ม (Reverse Segment)

การทำงาน: สุ่มเลือกรถมา 1 คัน สุ่มตัดช่วงของลูกค้า (เช่น เลือกลูกค้าคนที่ 2 ถึง 4 ในเส้นทาง) แล้วจับลำดับสลับจากหน้าไปหลัง

ในโค้ด: ใช้คำสั่ง `if method == "reverse_segment"`: เมื่อเช็คผ่าน จะพลิกอาร์เรย์ด้วย `r[i:j+1] = r[i:j+1][::-1]`



🚚 2. ย้ายลูกค้าข้ามคันแบบสุ่ม (Move Customer)

การทำงาน: สุ่มเลือกรถคันที่ 1 แล้วแอบดึงลูกค้าออกมา 1 คน จากนั้นนำไปแทรกแบบมั่วๆ ใส่ตรงไหนก็ได้ในรถคันที่ 2 (โดยมีข้อแม้ว่าคันที่ 2 จะต้องมีพื้นที่ Capacity เหลือพอ)

ในโค้ด: ใช้คำสั่ง `elif method == "move_customer":`



🔄 3. สลับลูกค้าคนละคันแบบสุ่ม (Swap Between)

การทำงาน: สุ่มเลือกรถมา 2 คัน เลือกลูกค้ามาคันละ 1 คน แล้วจับสลับที่กันดื้อๆ (โดยตรวจสอบว่าการสลับต้องไม่ทำให้น้ำหนักของรถคันใดคันหนึ่งเกินขีดจำกัด)

ในโค้ด: ใช้คำสั่ง `elif method == "swap_between":`



🧹 การล้างเส้นทาง & การขัดเกลา

1. ล้างรถที่ว่างเปล่า (Clean-up)

บางครั้งการสุ่มหยิบลูกค้าออกแบบ Move Customer อาจทำให้รถบางคันไม่มีลูกค้าเหลืออยู่เลย คือวิ่งออกจาก Depot แล้วกลับเข้า Depot ทันที (มีจำนวน Node ในเส้นทางแค่ 2 จุด)

```
# ทำความสะอาดและลบรถคันที่ว่างเปล่าทิ้งไป
new_routes = [r for r in new_routes if len(r) > 2]
```

2. ส่งกลับไปจัดระเบียบ (Re-Optimization)

เส้นทางที่เพิ่งถูกเขย่าเสร็จ มักจะแย่ลง ระยะทางไกลขึ้น และเส้นทางวิ่งตัดกันไปมา โค้ดจะส่งมันกลับไปให้ Stage 3 ดึงเส้นให้ตึงอีกรอบ

```
# ส่งไปทำ Local Search เต็มสูบอีกรอบ
perturbed_opt = full_optimization(perturbed)
```

ถ้าโชคดี ผลลัพธ์สุดท้ายสั้นกว่าเดิม แปลว่า **หลุดจากทางตันสำเร็จ!** 🎉





STAGE 5 :

VALIDATION





10 Strict Rules Checklist

👥 กฎการแวะลูกค้า

- 1. โหนดมีอยู่จริง (Nodes exist)
- 2. ส่งลูกค้าครบถ้วน (All served)
- 3. ไม่ส่งซ้ำซ้อน (No duplicates)

🚚 กฎข้อจำกัดของรถ

- 4. ไม่เกินความจุ (Capacity)
- 5. โหลดตรงกัน (Load match)
- 6. รถไม่เกินโควต้า (Vehicle count)

📍 กฎเส้นทางและระยะทาง

- 7. เริ่มต้นที่ Depot (Start at depot)
- 8. จบที่ Depot (End at depot)
- 9. ไม่แวะ Depot กลางทาง (No mid-depot)
- 10. ระยะทางรวมถูกต้อง (Distance match)

ตัวอย่างการเช็คเงื่อนไขในไฟล์ routeoptimization_version4.py ภายในฟังก์ชัน validate_routes:

```
def validate_routes(routes, depot, customers, demands, capacity, max_vehicles, ...):
    results = []
    all_pass = True

    # 4. ตรวจสอบ Capacity (ความจุรถ)
    cap_ok = True
    for i, route in enumerate(routes):
        # รวม demand ของทุกคนในคันนั้น
        rd = sum(demands[n] for n in route if n ≠ depot)
        if rd > capacity:
            results.append((f"4. Capacity (V{i+1})", "FAIL", f"Demand {rd} > {capacity}"))
            cap_ok = False
            all_pass = False

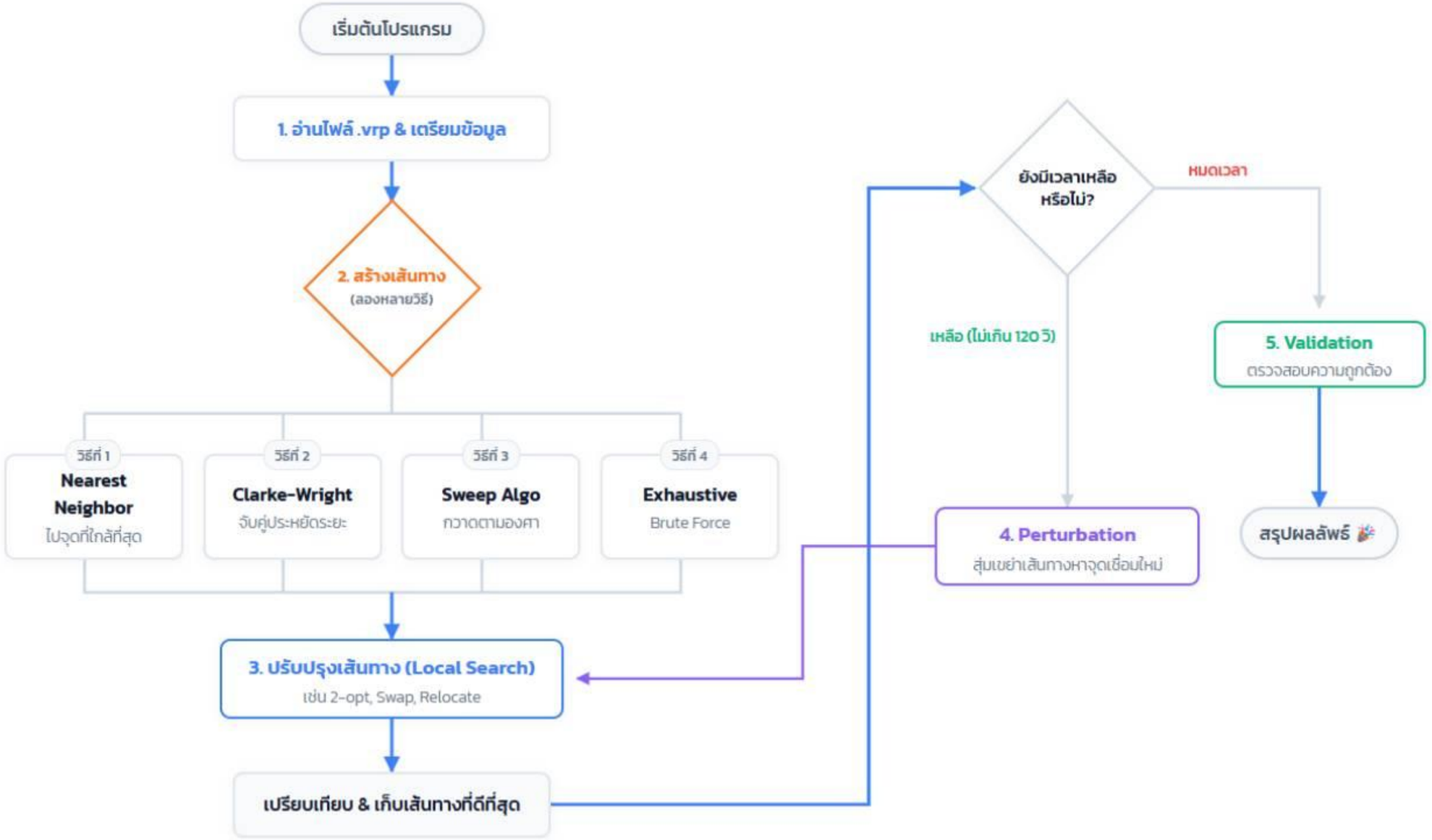
    # 6. ตรวจสอบว่าเส้นทางเริ่มที่ Depot หรือไม่
    bad_start = [i+1 for i, r in enumerate(routes) if r[0] ≠ depot]
    if bad_start:
        results.append(("6. Start at depot", "FAIL", f"ไม่ได้เริ่มที่ Depot: {bad_start}"))
        all_pass = False

    return results, all_pass
```

หาก all_pass คืนค่าเป็น True แปลว่าคำตอบนี้พร้อมใช้งาน 100%!

📌 โครงสร้างการทำงานของโปรแกรม CVRP

คลิกที่แต่ละขั้นตอน (กล่องข้อความ) ในแผนภาพเพื่อเรียนรู้รายละเอียดและอัลกอริทึมที่อยู่เบื้องหลัง







PROGRESSION FROM VERSION 1 TO VERSION 4





🛠️ Feature Progression

Feature Category	Version 1	Version 2	Version 3	Version 4
Construction Heuristics	Nearest Neighbor	+ Multi-start NN + Clarke-Wright + Exhaustive Partition (2V)	+ Sweep Algorithm (Angle-based) + Exhaustive Partition (3V)	*Same as v3*
Local Search (Intra-route)	❌ None	✅ 2-opt, Or-opt	✅ *Same as v2*	✅ *Same as v2*
Local Search (Inter-route)	❌ None	✅ Relocate, Swap, Cross-exchange	✅ Capped Cross-exchange (max_seg=4)	✅ *Same as v3*
Meta-heuristics	❌ None	❌ None	✅ Perturbation (Iterated Local Search)	✅ *Same as v3*
Execution Control	❌ None	❌ None	✅ Time Limits (`TIME_LIMIT=120`)	✅ *Same as v3*
Vehicle Calculation	⚠️ Fallback Value	⚠️ Fallback Value	✅ Auto-calculates Min Vehicles	✅ *Same as v3*
User Experience (UX)	🟢 Basic Validation	🟢 Compares Strategies	🟢 *Same as v2*	🌟 Refined Output & Smart Skip Messages





BENCHMARK RESULTS





รู้จัก 3 Benchmark

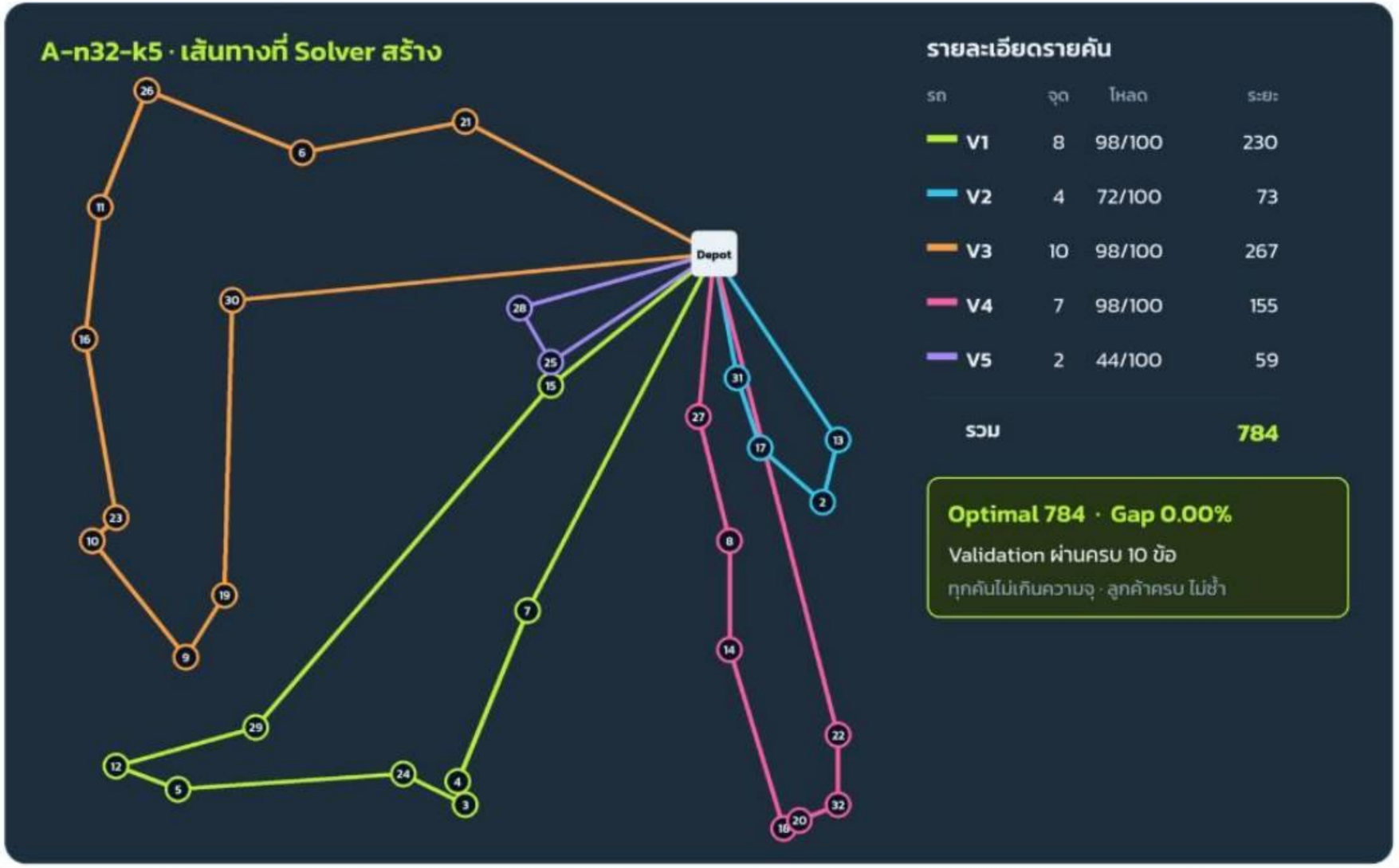
หัวข้อ: 3 Benchmark ที่ใช้ทดสอบ Solver

	A-n32-k5	B-n31-k5	P-n40-k5
ลูกค้า	31	30	39
ความจุต่อคัน	100	100	140
Demand รวม	410	412	618
รถ	5 คัน	5 คัน	5 คัน
ใช้ความจุรวม	82%	82%	88%
ลักษณะ	ลูกค้ากระจายสุ่ม	ลูกค้าเป็นกลุ่ม	ใหญ่สุด ความจุตึงสุด

Result Card: A-n32-k5

ผลลัพธ์การจัดเส้นทางสำหรับชุดโจทย์ A-n32-k5

วิธี: NN seed 7 + Local Search | เวลา: 5.3 วินาที



รายละเอียดรายคัน

รถ	จุด	โหลด	ระยะ
V1	8	98/100	230
V2	4	72/100	73
V3	10	98/100	267
V4	7	98/100	155
V5	2	44/100	59
รวม			784

Optimal 784 · Gap 0.00%

Validation ผ่านครบ 10 ข้อ

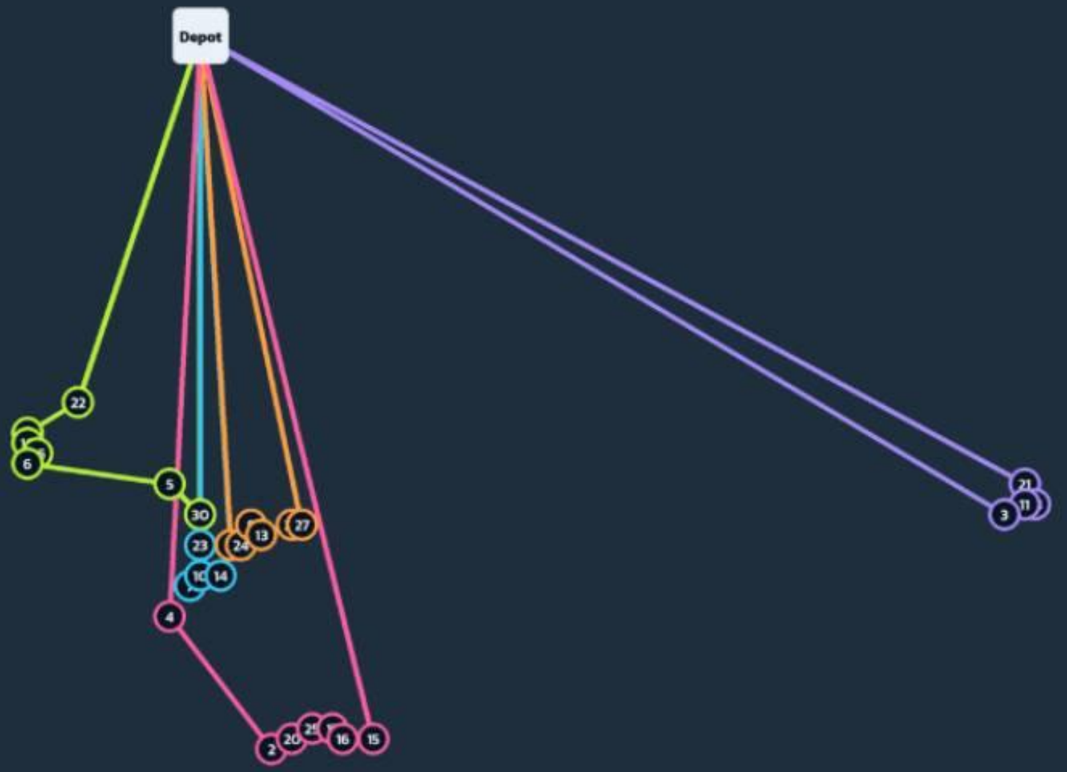
ทุกคันไม่เกินความจุ · ลูกค้าครบ ไม่ซ้ำ

Result Card: B-n31-k5

ผลลัพธ์การจัดเส้นทางสำหรับชุดโจทย์ B-n31-k5

วิธี: NN + Local Search | เวลา: 5.2 วินาที

B-n31-k5 · เส้นทางที่ Solver สร้าง



รายละเอียดรายคัน

รถ	จุด	โหลด	ระยะ
V1	7	86/100	112
V2	6	96/100	110
V3	6	97/100	107
V4	7	95/100	154
V5	4	38/100	189
รวม			672

Optimal 672 · Gap 0.00%

Validation ผ่านครบ 10 ข้อ

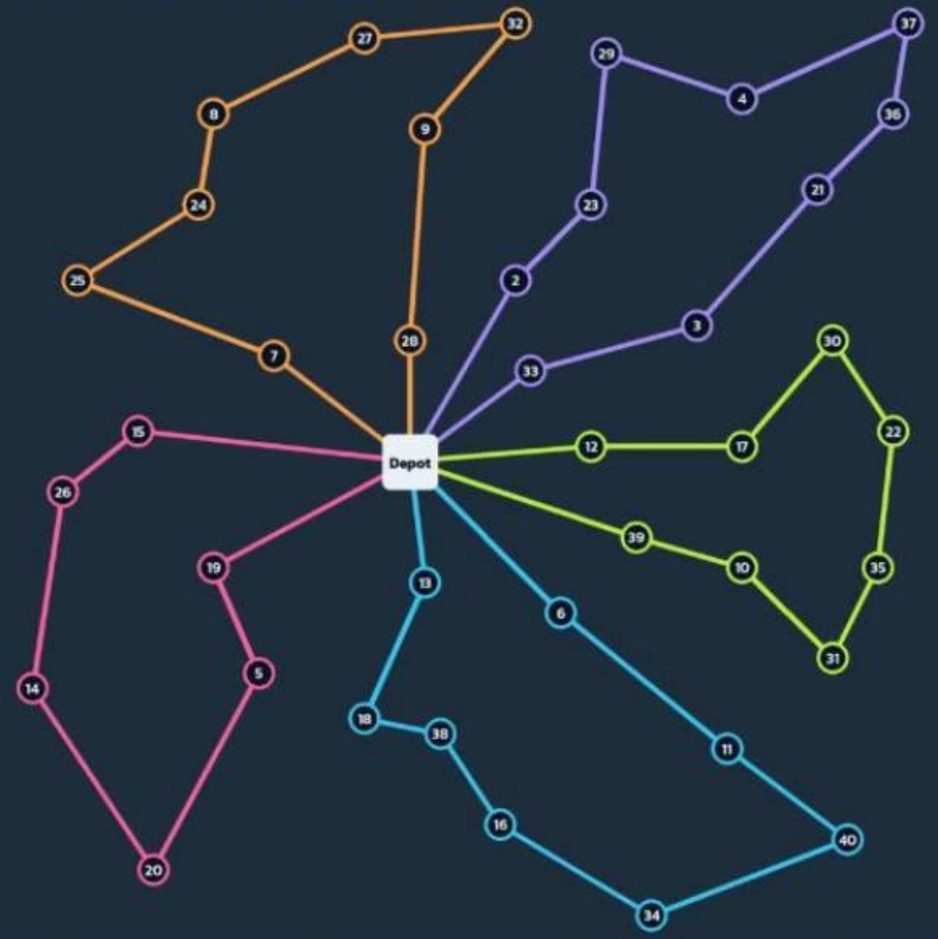
ทุกคันไม่เกินความจุ · ลูกค้าครบ ไม่ซ้ำ

Result Card: P-n40-k5

ผลลัพธ์การจัดเส้นทางสำหรับชุดโจทย์ P-n40-k5

วิธี: NN seed 6 + Local Search | เวลา: 12.9 วินาที

P-n40-k5 · เส้นทางที่ Solver สร้าง



รายละเอียดรายคัน

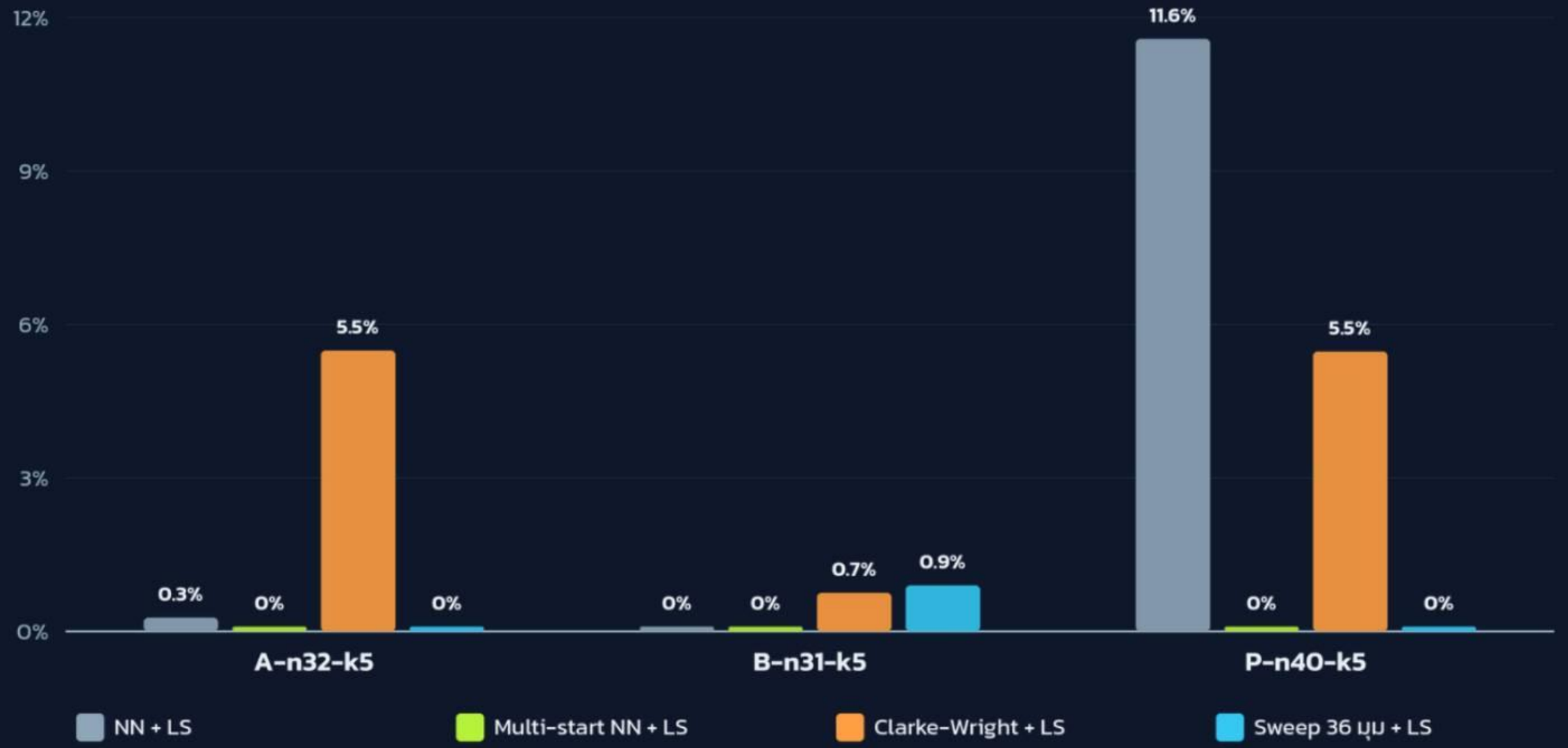
รถ	จุด	โหลด	ระยะ
V1	8	119/140	85
V2	8	114/140	94
V3	8	116/140	92
V4	6	131/140	89
V5	9	138/140	98
รวม			458

Optimal 458 · Gap 0.00%

Validation ผ่านครบ 10 ข้อ

ทุกคันไม่เกินความจุ · ลูกค้าครบ ไม่ซ้ำ

Gap จาก Optimal ของแต่ละวิธีสร้างเส้นทาง (หลัง Local Search แล้ว)



สรุปผลและข้อสังเกต

หัวข้อ: ผลรวม 3 Benchmark

	A-n32-k5	B-n31-k5	P-n40-k5
ลูกค้า · ความจุ	31 · 100	30 · 100	39 · 140
ระยะที่ได้	**784**	**672**	**458**
Optimal	784	672	458
Gap	**0.00%**	**0.00%**	**0.00%**
วิธีที่ได้ผลดีสุด	NN seed 7 + LS	NN + LS	NN seed 6 + LS
เวลา	5.3 วิ	5.2 วิ	12.9 วิ
Validation	ผ่าน 10 ข้อ	ผ่าน 10 ข้อ	ผ่าน 10 ข้อ

🔍 **ข้อสังเกต 3 ข้อ:**

1. Gap 0.00% ทั้ง 3 ชุด ผ่าน Validation ครบ 10 ข้อ
2. Perturbation 50 รอบไม่พบการปรับปรุงเพิ่มในทุกชุด เพราะ Local Search ไปถึงค่าที่ดีที่สุดแล้ว
3. Exhaustive partition ไม่ได้ทำงานในทั้ง 3 ชุด เพราะลูกค้าเกิน 22 จุด

การจัดเส้นทางส่งหินและทรายของลานวัสดุ CPAC

Self-created Case · Mission 3

AI Hackathon Route Optimization 2026

ใช้ CVRP จัดรถให้สังครบทุกไซต์
ด้วยระยะทางและการปล่อยคาร์บอนที่ลดลง

ข้อมูลกรณีศึกษา: **20** จุดส่ง | **83** ตัน | **8** คัน



การจัดรถเดิมใช้ความจุขาออกเพียง 79%



ลานม่รถดัมพ์ 10 ล้อ
8 คัน

ในสถานการณ์ของโจทย์
ออเดอร์หลายไซต์เข้าพร้อมกัน
จนต้องเลื่อนหรือปฏิเสธงานบางรายการ

ความจุรวมของรถที่ออกงาน
เท่ากับ **7 × 15 = 105 ตัน**

อัตราบรรทุกตอนออกจากลาน
= 83 ÷ 105 × 100 ≈ 79%

ความจุขณะออกจากลาน

79%

> **การจัดกลุ่มออเดอร์ใหม่อาจช่วยให้ใช้รถต่อรอบน้อยลง**

เหตุผลที่เลือกกรณีส่งหินและทราย

กรณีศึกษาใช้บริบทธุรกิจวัสดุก่อสร้างของ CPAC ในเครือ SCG โดยเลือกการส่งหินและทราย





ประเด็น	คอนกรีตผสมเสร็จ	หินและทรายในกรณีนี้
ลักษณะสินค้า	มีข้อจำกัดด้านเวลาระหว่างผลิตและส่งมอบ	ไม่มีข้อจำกัดด้านเวลาแข็งตัว
เงื่อนไขสำคัญ	ต้องพิจารณาตารางผลิตและส่งมอบ ควบคู่เส้นทาง	เน้นความจุรถและระยะทาง
แบบจำลองที่เกี่ยวข้อง	Concrete Delivery Problem	Capacitated Vehicle Routing Problem

หินและทรายเหมาะกับการใช้ CVRP ในขอบเขตของกรณีศึกษานี้

ลานลาดกระบังให้บริการ 20 จุด



จุดเริ่มต้น:
ลานวัสดุย่านลาดกระบัง

พื้นที่ให้บริการ:
กรุงเทพตะวันออกและสมุทรปราการ



ครอบคลุมคลังสินค้ารอบสุวรรณภูมิ
คอนโดตามแนวบางนา และบ้านจัดสรร
ฝั่งมีนบุรี–หนองจอก

ระยะตรงจากลานถึงจุดส่ง:
2.8–22.1 กม.

Depot 1 และลูกค้า 20 จุด รวม
21 Nodes



ภาพแนวคิดพื้นที่ให้บริการ ไม่ใช่แผนที่พิกัดจริง

ขั้นตอนการทำงาน: Solver หาเส้นทาง แล้วคำนวณคาร์บอนภายหลัง

1



ข้อมูลโจทย์

20 จุด · 83 ตัน
รถ 15 ตัน · 8 คัน

2



Solver

หาเส้นทางที่
ระยะทางรวมสั้นที่สุด

3



เส้นทาง 6 คัน

ได้ระยะรายช่วงและ
น้ำหนักบนรถรายช่วง

4



คำนวณคาร์บอน

ระยะ × ค่าปล่อยต่อ กม.
ตามน้ำหนัก (TGO)

5



เทียบกับ FIFO

ใช้วิธีคำนวณเดียวกัน
ทั้งสองฝั่ง

Solver ไม่ได้คำนวณคาร์บอน — คาร์บอนคำนวณจากเส้นทางที่ Solver ให้มา

แบบจำลองกำหนดความจุรถ 15 ตัน

น้ำหนักรวม **25 ตัน** – น้ำหนักรถเปล่าโดยประมาณ **10 ตัน** = น้ำหนักสินค้า **15 ตัน**

พิกัดน้ำหนักรวม GVW ที่เอกสารอ้างถึง

25 ตัน – ประมาณ 10 ตัน = 15 ตัน



องค์ประกอบ	ค่าที่ใช้ในกรณีศึกษา
พิกัดน้ำหนักรวม GVW ที่เอกสารอ้างถึง	25 ตัน
น้ำหนักรถเปล่าโดยประมาณ	10 ตัน
ความจุสินค้าของรถแต่ละคัน	15 ตัน

รถที่ใช้ในแบบจำลองเป็นรถ 10 ล้อ ความจุเท่ากันทั้งหมด **8 คัน**



หมายเหตุ: ค่าตามสมมติฐานและที่มาที่ระบุในเอกสารกรณีศึกษา

Demand สะท้อนประเภทงานก่อสร้าง

กำหนด Demand ตามลักษณะงาน ไม่ได้สุ่มตัวเลข

ประเภทงาน	จำนวนจุด	ตันต่อจุด	Demand รวม	เหตุผล
คอนโด / อาคารสูง	5	5 – 6	**28 ตัน**	งานพื้นและฐานรากปริมาณมาก
โกดัง / คลังสินค้า	4	5 – 6	**22 ตัน**	พื้นที่กว้าง ใช้วัสดุมาก
บ้านจัดสรร	5	3 – 4	**18 ตัน**	ทยอยก่อสร้างทีละหลัง
อาคารพาณิชย์	3	3	**9 ตัน**	พื้นที่จำกัด ส่งครั้งละไม่มาก
งานซ่อมถนน / ทางเท้า	3	2	**6 ตัน**	ปริมาณงานน้อย
รวม	**20**	–	**83 ตัน**	–

Demand รวม

83 ตัน



หมายเหตุ: จำนวนจุดและยอดรวมตรวจนับจากตาราง Node 2–21

แปลงพิกัดให้ Solver คำนวณเป็นกิโลเมตร

ใช้ละติจูดและลองจิจูดของพื้นที่ แล้วกำหนดลานวัสดุเป็นจุดกำเนิด (0, 0)



$$x = (\text{lon} - \text{lon}_0) \times 108.14$$

$$y = (\text{lat} - \text{lat}_0) \times 110.57$$

- x และ y มีหน่วยกิโลเมตร
- $108.14 \approx 111.32 \times \cos(13.727°)$
- Solver ใช้ระยะทางแบบ EUC_2D
- ไม่ใช้ค่าองศาเป็นระยะทางโดยตรง เพราะ 1 องศาละติจูดและลองจิจูด มีความยาวไม่เท่ากัน

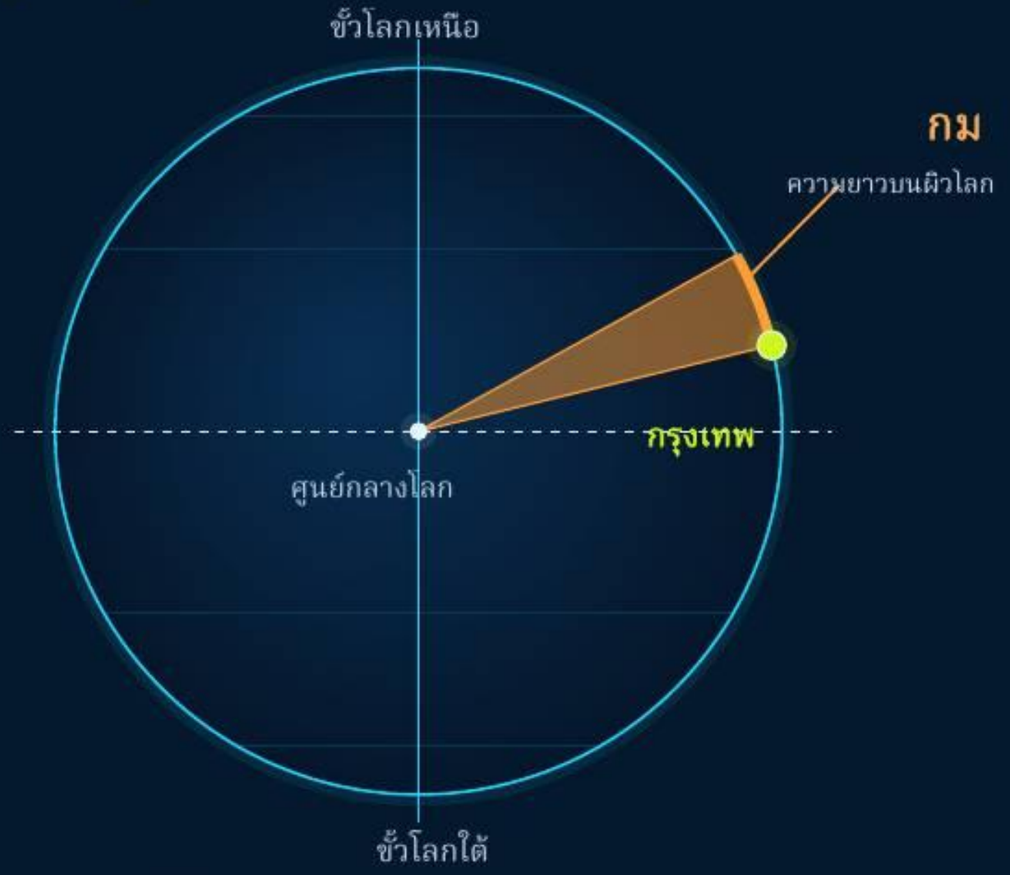


ที่มาของ และ อธิบายในหน้าถัดไป

ที่มาของ องศาแนวเหนือ ไต้

ละติจูดและลองจิจูดเป็นมุม ไม่ใช่ระยะทาง จึงต้องแปลงเป็นกิโลเมตรก่อน

ผ่าลูกโลกดูจากด้านข้าง



มุมในภาพขยายเพื่อให้เห็นชัด

ทำไม องศา กม

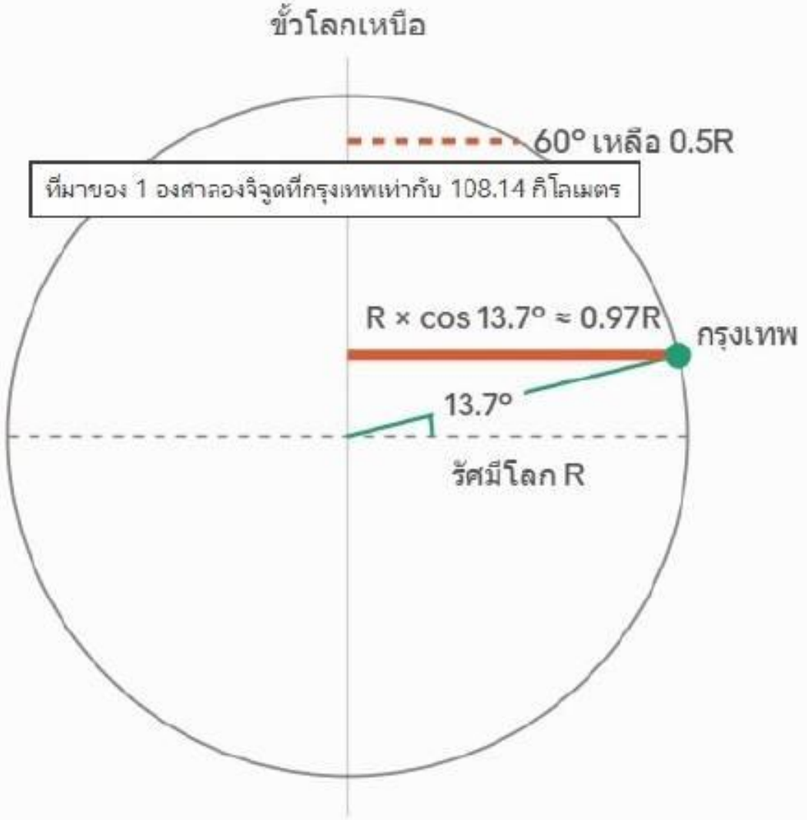
- **รอบโลกแนวเหนือ ไต้**
  ยาวประมาณ กม
- **วงกลมแบ่งได้ องศา**
- กม ต่อ องศา
- **โลกแบนเล็กน้อยที่ขั้ว**
  ใกล้เส้นศูนย์สูตร กม

ตัวอย่าง ลาน จุด มีนบุรี

ละติจูดต่างกัน องศา

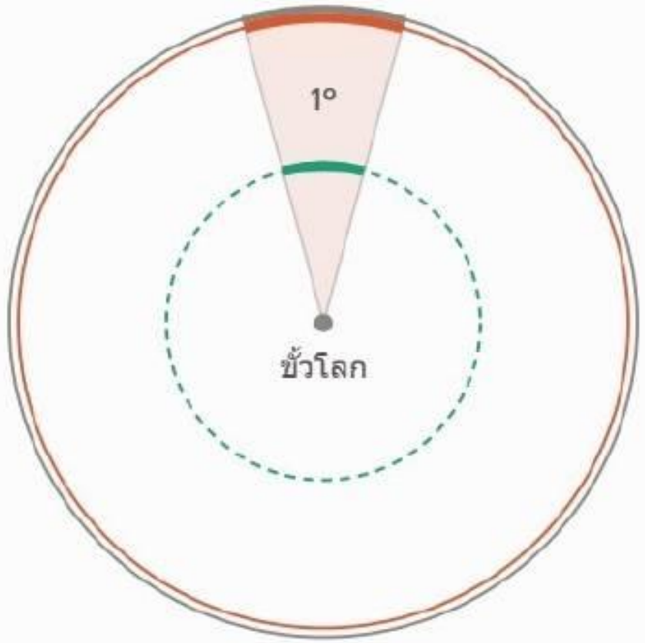
กม ไปทางเหนือ

มองด้านข้าง



ยิ่งขึ้นเหนือ วงรอบโลกยิ่งเล็ก

มองจากขั้วโลกเหนือลงมา



เส้นศูนย์สูตร 0° · 1° = 111.32 กม.

กรุงเทพ 13.7° · 1° = 108.14 กม.

ละติจูด 60° · 1° = 55.66 กม.

รอบเส้นศูนย์สูตร 40,075 ÷ 360 = 111.32

× cos 13.727° (0.9714) = 108.14 กม.

ที่มาของ องศาแนวตะวันออก ตะวันตก

ยิ่งขึ้นไปทางเหนือ วงรอบโลกยิ่งเล็ก จึงต้องคูณด้วย ของละติจูด

เส้นรอบวงโลกตามแนวเส้นศูนย์สูตร

กม

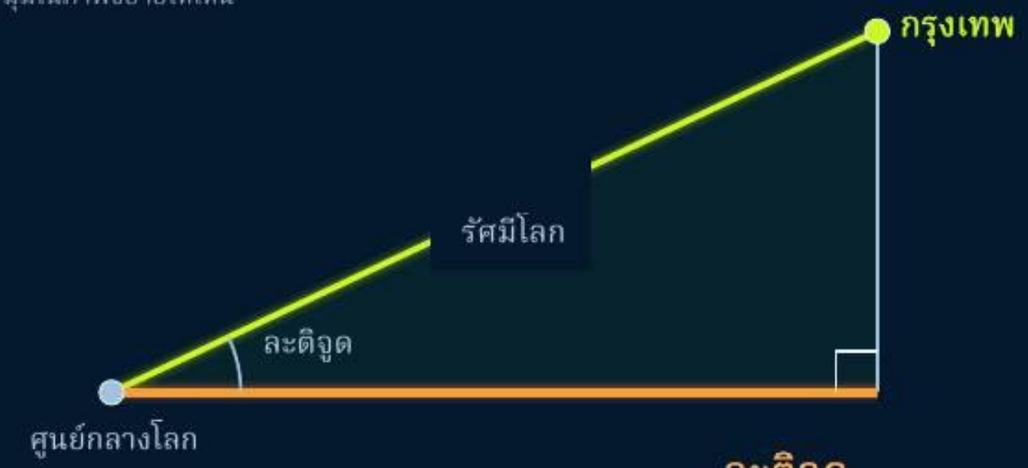
หารทั้งสองข้างด้วย เพื่อหารัศมีโลก

$$r = \frac{C}{2\pi} = \frac{40\,075}{2\pi} \approx 6\,378.13434 \text{ km}$$

สามเหลี่ยมในลูกโลก

ด้านประชิดมุม ด้านยาวที่สุด

มุมในภาพขยายให้เห็น



ละติจูด

ระยะจากแกนโลก รัศมีของวงที่กรุงเทพ

ไล่ตัวเลขทีละขั้น

รัศมีโลก

กม

รัศมีของวงที่กรุงเทพ

กม

สูตรเส้นรอบวง

วงรอบโลกที่กรุงเทพ

กม

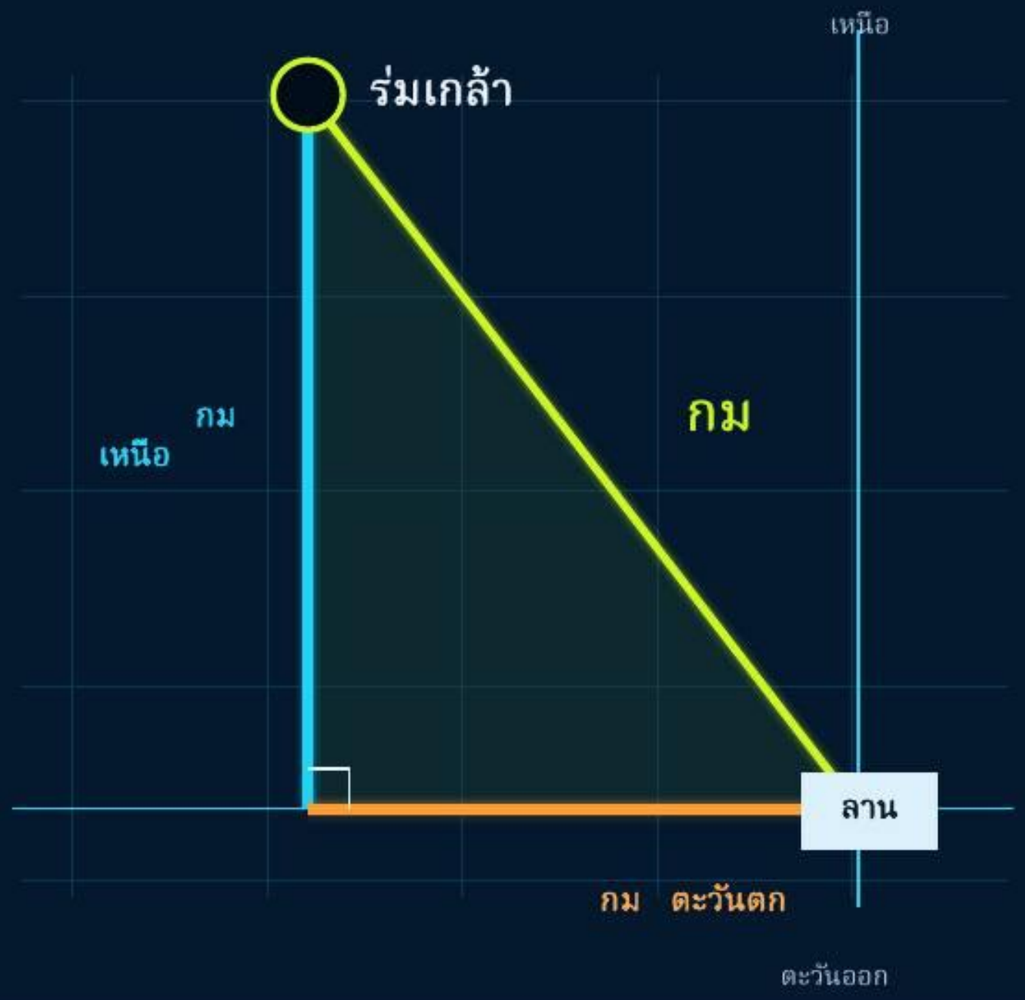
องศา

องศาลองจิจูดที่กรุงเทพ

กม

วัดระยะแบบพีทาโกรัส

ตัวอย่างจุด ร่มเกล้า จากพิกัดจริงถึงระยะจากลาน



ระยะแบบพีทาโกรัส

ขั้นที่ แปลงพิกัดจุด

จุดที่สนใจ ลาน เสมอ

ขั้นที่ หาระยะเส้นตรง

ระยะ

กม

ตรงกับระยะจากลานของจุด ในตารางข้อมูล

ระยะนี้เป็นเส้นตรง ไม่ใช่ระยะตามถนน

ข้อมูลจุดส่ง

Depot คือ Node 1 พิกัด (0, 0) และ Demand 0 ตัน

Node	จุดให้บริการ	ประเภทงาน	Demand (ตัน)	x (กม.)	y (กม.)	ระยะตรงจากลาน (กม.)
2	ร่มเกล้า คอนโดฯ	คอนโด/อาคารสูง	6	−2.81	3.65	4.6
3	เคหะร่มเกล้า ปรับปรุงถนน	งานซ่อมถนน	2	−1.62	2.32	2.8
4	ลาดกระบัง อาคารพาณิชย์	อาคารพาณิชย์	3	−2.92	−0.55	3.0
5	กิ่งแก้ว โกดัง	โกดัง/คลังสินค้า	6	−4.43	−3.54	5.7
6	บางพลี โรงงาน	โกดัง/คลังสินค้า	5	−4.43	−13.49	14.2

ข้อมูลจุดส่ง

Node	จุดให้บริการ	ประเภทงาน	Demand (ตัน)	x (กม.)	y (กม.)	ระยะตรงจากลาน (กม.)
7	สุวรรณภูมิ คลังสินค้า	โกดัง/คลังสินค้า	6	−6.38	−3.21	7.1
8	บางนา คอนโดฯ	คอนโด/อาคารสูง	5	−15.03	−6.52	16.4
9	ศรีนครินทร์ อาคารสำนักงาน	คอนโด/อาคารสูง	5	−15.25	−4.09	15.8
10	อ่อนนุช บ้านจัดสรร	บ้านจัดสรร	4	−20.01	−2.32	20.1
11	ประเวศ บ้านจัดสรร	บ้านจัดสรร	3	−13.09	−1.22	13.1

ข้อมูลจุดส่ง

Node	จุดให้บริการ	ประเภทงาน	Demand (ตัน)	x (กม.)	y (กม.)	ระยะตรงจากลาน (กม.)
12	พระโขนง ซ่อมทางเค้า	งานซ่อมถนน	2	−20.01	−3.32	20.3
13	สุขุมวิท 101 คอนโดฯ	คอนโด/อาคารสูง	6	−20.44	−3.87	20.8
14	สำโรง อาคารพาณิชย์	อาคารพาณิชย์	3	−20.33	−8.73	**22.1** ไกลสุดตามตารางข้อมูล
15	เทพารักษ์ บ้านจัดสรร	บ้านจัดสรร	4	−16.33	−11.28	19.8
16	บางบ่อ โกดัง	โกดัง/คลังสินค้า	5	7.03	−15.26	16.8

ข้อมูลจุดส่ง

Node	จุดให้บริการ	ประเภทงาน	Demand (ตัน)	x (กม.)	y (กม.)	ระยะตรงจากลาน (กม.)
17	มีนบุรี บ้านจัดสรร	บ้านจัดสรร	4	−4.11	9.51	10.4
18	คลองสองมวา บ้านจัดสรร	บ้านจัดสรร	3	−7.79	14.26	16.3
19	หนองจอก ซ่อมถนน	งานซ่อมถนน	2	8.22	14.26	16.5
20	รามคำแหง คอนโดฯ	คอนโด/อาคารสูง	6	−15.46	4.31	16.1
21	นวมินทร์ อาคารพาณิชย์	อาคารพาณิชย์	3	−15.79	9.51	18.4

รวมข้อมูล Node 2–21: ลูกค้า 20 จุด และ Demand 83 ตัน

เงื่อนไขที่ทุกคำตอบต้องผ่าน

01



ความจุ

สินค้ารวมต่อคัน
ไม่เกิน 15 ตัน

02



จำนวนรถ

ใช้ได้ไม่เกิน
8 คัน

03



การให้บริการ

ครบทุกจุด
จุดละ 1 ครั้ง
จากรถคันเดียว

04



ต้นทางและปลายทาง

ทุกเส้นทางเริ่มและจบ
ที่ลานวัสดุ

05



ประเภทรถ

รถ 10 ล้อความจุ
เท่ากันทุกคัน

เป้าหมายของ Solver:
ลดระยะทางรวมภายใต้เงื่อนไขเหล่านี้

เตรียมข้อมูลเป็น CVRPLIB

ใช้รูปแบบอินพุตเดียวกับ Solver ใน Mission 2

CPAC-AGG-LKB-20.vrp (ตัวอย่างโครงสร้าง)

```
NAME :      CPAC-AGG-LKB-20
COMMENT :  (Aggregate delivery, Lat Krabang yard,
            10-wheel dump truck, No of trucks: 8)
TYPE :      CVRP
DIMENSION : 21
EDGE_WEIGHT_TYPE : EUC_2D
CAPACITY : 15

NODE_COORD_SECTION ...
DEMAND_SECTION ...
DEPOT_SECTION ...
...
```

นี่เป็นตัวอย่างโครงสร้างไฟล์ ไม่ใช่ไฟล์อินพุตฉบับเต็มที่พร้อมรัน
ข้อมูลครบทุก ดูหน้า

อ้างอิง
ข้อมูลครบทุก ดูหน้า

21 Nodes
ประกอบด้วย Depot 1 จุด และจุดส่ง 20 จุด

15 ตัน
ความจุรถต่อคัน (CAPACITY) กำหนดเป็น 15 ตัน

8 คัน
กำหนดรถพร้อมใช้เป็น 8 คัน ตามโจทย์



ให้จำนวนรถที่ใช้จริง
เป็นผลลัพธ์จากการจัดเส้นทาง

```
NAME : CPAC-AGG-LKB-20
COMMENT : (Aggregate delivery, Lat Krabang yard, 10-wheel dump truck, No of trucks: 8)
TYPE : CVRP
DIMENSION : 21
EDGE_WEIGHT_TYPE : EUC_2D
CAPACITY : 15
NODE_COORD_SECTION
 1 0.000 0.000
 2 -2.812 3.649
 3 -1.622 2.322
 4 -2.920 -0.553
 5 -4.434 -3.538
 6 -4.434 -13.490
 7 -6.380 -3.207
 8 -15.032 -6.524
 9 -15.248 -4.091
 10 -20.006 -2.322
 11 -13.085 -1.216
 12 -20.006 -3.317
 13 -20.439 -3.870
 14 -20.330 -8.735
 15 -16.329 -11.278
 16 7.029 -15.259
 17 -4.109 9.509
 18 -7.786 14.264
 19 8.219 14.264
 20 -15.464 4.312
 21 -15.789 9.509
DEMAND_SECTION
 1 0
 2 6
 3 2
 4 3
 5 6
 6 5
```

```
DEMAND_SECTION
 1 0
 2 6
 3 2
 4 3
 5 6
 6 5
 7 6
 8 5
 9 5
 10 4
 11 3
 12 2
 13 6
 14 3
 15 4
 16 5
 17 4
 18 3
 19 2
 20 6
 21 3
DEPOT_SECTION
 1
 -1
EOF
```

Solver สร้างคำตอบแล้วปรับปรุงเส้นทาง

ขั้นที่ 1: สร้างเส้นทางเริ่มต้นหลายวิธี

- Nearest Neighbor
- Clarke-Wright Savings
- Sweep

เปรียบเทียบและเลือกคำตอบที่ดีที่สุด

ขั้นที่ 2: ปรับปรุงเส้นทาง

- 2-opt และ Or-opt
- Relocate, Swap และ Cross-exchange
- Perturbation
  เพื่อสำรวจคำตอบเพิ่มเติม
  และลดโอกาสติด Local Optimum

Local Search

Best tested solution



ใช้ Solver ชุดเดียวกับ Benchmark ทั้ง 3 ชุด
โดยไม่ปรับแต่งเฉพาะกรณีนี้

การทดสอบรูปแบบเส้นทางทั้งหมดของ 20 จุดมีจำนวนมาก
จึงใช้วิธี **Heuristic**

FIFO เป็นเส้นฐานจำลองสำหรับเปรียบเทียบ

การจำลองแบบ FIFO

1. ไม่มีข้อมูลเส้นทางเดิมของลานวัสดุจริง
2. จำลองการจ่ายรถตามลำดับที่ลูกค้าติดต่อเข้ามา
3. จัดออเดอร์ตามความจุ แล้วขึ้นรถคันใหม่เมื่อคันเดิมไม่สามารถรับออเดอร์ถัดไปได้
4. ใช้เส้นฐานนี้เปรียบเทียบกับผลจาก Solver

แนวคิดการจ่ายรถแบบ FIFO





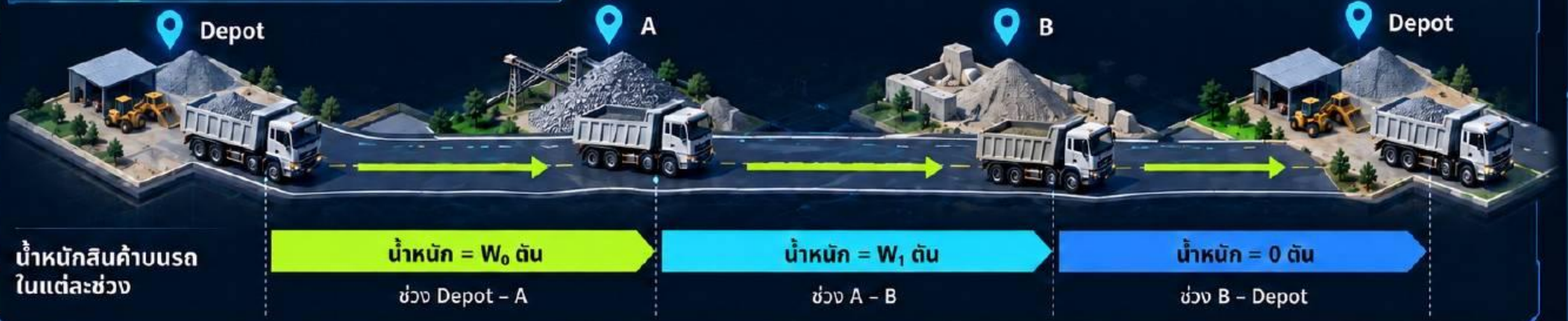
ผลของ FIFO: ใช้รถ **7** คัน | ระยะทางรวม **302.9** กม.

ทำไมเส้นฐานไม่ใช่เหมือนใน

คือขั้นแรกข้างในแทนวิธีที่ธุรกิจจ่ายรถจริง จึงวัดได้ว่าช่วยธุรกิจได้เท่าไร

ตัวชี้วัดคำนวณจากเส้นทางและน้ำหนักรายช่วง

ตัวอย่างวิธีคำนวณ: เส้นทาง Depot – A – B – Depot



ระยะทางของช่วง A–B

$$d(A,B) = \sqrt{[(x_A - x_B)^2 + (y_A - y_B)^2]}$$

ระยะทางรวม

= ผลรวมระยะทุกช่วงของทุกเส้นทาง

คาร์บอน

$$\sum_{ij}(\text{ระยะช่วง} \times \text{ปัจจัยการปล่อยต่อกม. ตามน้ำหนักช่วงนั้น})$$

Solver ลดระยะทางรวม ส่วนคาร์บอนคำนวณเพิ่มจากเส้นทางที่ได้

แปลงปัจจัยการปล่อยให้เป็นหน่วยต่อกิโลเมตร

ข้อมูลอ้างอิงในกรณีศึกษา: รถบรรทุก 10 ล้อ ดีเซล วิ่งปกติ น้ำหนักบรรทุกสูงสุด 16 ตัน

อัตราบรรทุก	น้ำหนักสินค้า	ปัจจัยจากแหล่งข้อมูล	แปลงเป็น $kgCO_2e$/กม.
0%	0 ตัน	0.5744 $kgCO_2e$/กม.	**0.5744**
50%	8 ตัน	0.0852 $kgCO_2e$/ตัน-กม.	0.0852 × 8 = **0.6816**
75%	12 ตัน	0.0589 $kgCO_2e$/ตัน-กม.	0.0589 × 12 = **0.7068**
100%	16 ตัน	0.0454 $kgCO_2e$/ตัน-กม.	0.0454 × 16 = **0.7264**



แหล่งที่เอกสารระบุ:
Thai National LCI Database,
TIIS–MTEC–NSTDA ร่วมกับ TGO, Update April 2026

ตัวที่มีการเปลี่ยนแปลง ตัวที่มีการเพิ่มเติม UPDATE: 6 กรกฎาคม 2569

ลำดับที่	ชื่อ	รายละเอียด	หน่วย	ค่าแฟคเตอร์ ($kgCO_2e$/หน่วย)	แหล่งข้อมูลอ้างอิง	วันที่อัพเดท
101.	รถกระบะบรรทุก 6 ล้อ ขนาดใหญ่ วิ่งปกติ 75% Loading	น้ำหนักบรรทุกสูงสุด 11 ตัน; ใช้น้ำมันดีเซล เป็นเชื้อเพลิง; LCIA method IPCC 2013 GWP 100a V1.03	tkm	0.0768	Thai National LCI Database, TIIS-MTEC-NSTDA (with TGO electricity 2022-2024)	Update_April2026
102.	รถกระบะบรรทุก 6 ล้อ ขนาดใหญ่ วิ่งปกติ 100% Loading	น้ำหนักบรรทุกสูงสุด 11 ตัน; ใช้น้ำมันดีเซล เป็นเชื้อเพลิง; LCIA method IPCC 2013 GWP 100a V1.03	tkm	0.0613	Thai National LCI Database, TIIS-MTEC-NSTDA (with TGO electricity 2022-2024)	Update_April2026
103.	รถกระบะบรรทุก 6 ล้อ ขนาดใหญ่ วิ่งแบบสมบุกสมบัน 0% Loading	น้ำหนักบรรทุกสูงสุด 11 ตัน; ใช้น้ำมันดีเซล เป็นเชื้อเพลิง; LCIA method IPCC 2013 GWP 100a V1.03	km	0.6079	Thai National LCI Database, TIIS-MTEC-NSTDA (with TGO electricity 2022-2024)	Update_April2026
104.	รถกระบะบรรทุก 6 ล้อ ขนาดใหญ่ วิ่งแบบสมบุกสมบัน 50% Loading	น้ำหนักบรรทุกสูงสุด 11 ตัน; ใช้น้ำมันดีเซล เป็นเชื้อเพลิง; LCIA method IPCC 2013 GWP 100a V1.03	tkm	0.1345	Thai National LCI Database, TIIS-MTEC-NSTDA (with TGO electricity 2022-2024)	Update_April2026
105.	รถกระบะบรรทุก 6 ล้อ ขนาดใหญ่ วิ่งแบบสมบุกสมบัน 75% Loading	น้ำหนักบรรทุกสูงสุด 11 ตัน; ใช้น้ำมันดีเซล เป็นเชื้อเพลิง; LCIA method IPCC 2013 GWP 100a V1.03	tkm	0.0942	Thai National LCI Database, TIIS-MTEC-NSTDA (with TGO electricity 2022-2024)	Update_April2026
106.	รถกระบะบรรทุก 6 ล้อ ขนาดใหญ่ วิ่งแบบสมบุกสมบัน 100% Loading	น้ำหนักบรรทุกสูงสุด 11 ตัน; ใช้น้ำมันดีเซล เป็นเชื้อเพลิง; LCIA method IPCC 2013 GWP 100a V1.03	tkm	0.0734	Thai National LCI Database, TIIS-MTEC-NSTDA (with TGO electricity 2022-2024)	Update_April2026
107.	รถตู้บรรทุก 10 ล้อ วิ่งปกติ 0% Loading	น้ำหนักบรรทุกสูงสุด 16 ตัน; ใช้น้ำมันดีเซล เป็นเชื้อเพลิง; LCIA method IPCC 2013 GWP 100a V1.03	km	0.5744	Thai National LCI Database, TIIS-MTEC-NSTDA (with TGO electricity 2022-2024)	Update_April2026
108.	รถตู้บรรทุก 10 ล้อ วิ่งปกติ 50% Loading	น้ำหนักบรรทุกสูงสุด 16 ตัน; ใช้น้ำมันดีเซล เป็นเชื้อเพลิง; LCIA method IPCC 2013 GWP 100a V1.03	tkm	0.0852	Thai National LCI Database, TIIS-MTEC-NSTDA (with TGO electricity 2022-2024)	Update_April2026
109.	รถตู้บรรทุก 10 ล้อ วิ่งปกติ 75% Loading	น้ำหนักบรรทุกสูงสุด 16 ตัน; ใช้น้ำมันดีเซล เป็นเชื้อเพลิง; LCIA method IPCC 2013 GWP 100a V1.03	tkm	0.0589	Thai National LCI Database, TIIS-MTEC-NSTDA (with TGO electricity 2022-2024)	Update_April2026
110.	รถตู้บรรทุก 10 ล้อ วิ่งปกติ 100% Loading	น้ำหนักบรรทุกสูงสุด 16 ตัน; ใช้น้ำมันดีเซล เป็นเชื้อเพลิง; LCIA method IPCC 2013 GWP 100a V1.03	tkm	0.0454	Thai National LCI Database, TIIS-MTEC-NSTDA (with TGO electricity 2022-2024)	Update_April2026
111.	รถตู้บรรทุก 10 ล้อ วิ่งแบบสมบุกสมบัน 0% Loading	น้ำหนักบรรทุกสูงสุด 16 ตัน; ใช้น้ำมันดีเซล เป็นเชื้อเพลิง; LCIA method IPCC 2013 GWP 100a V1.03	km	0.6776	Thai National LCI Database, TIIS-MTEC-NSTDA (with TGO electricity 2022-2024)	Update_April2026
112.	รถตู้บรรทุก 10 ล้อ วิ่งแบบสมบุกสมบัน 50% Loading	น้ำหนักบรรทุกสูงสุด 16 ตัน; ใช้น้ำมันดีเซล เป็นเชื้อเพลิง; LCIA method IPCC 2013 GWP 100a V1.03	tkm	0.1043	Thai National LCI Database, TIIS-MTEC-NSTDA (with TGO electricity 2022-2024)	Update_April2026
113.	รถตู้บรรทุก 10 ล้อ วิ่งแบบสมบุกสมบัน 75% Loading	น้ำหนักบรรทุกสูงสุด 16 ตัน; ใช้น้ำมันดีเซล เป็นเชื้อเพลิง; LCIA method IPCC 2013 GWP 100a V1.03	tkm	0.0724	Thai National LCI Database, TIIS-MTEC-NSTDA (with TGO electricity 2022-2024)	Update_April2026

ประมาณค่าการปล่อยของน้ำหนักระหว่างระดับ

ให้ค่าไว้ ระดับ คือ ตัน น้ำหนักอื่นใช้การประมาณค่าแบบเส้นตรง

สูตรประมาณค่า

ค่า ค่าจุดล่าง น้ำหนัก น้ำหนักจุดล่าง
อัตราเพิ่มต่อตัน

อัตราเพิ่มต่อตันของแต่ละช่วง

ช่วงน้ำหนัก	การคำนวณ	เพิ่มต่อ ตัน
ตัน		
ตัน		
ตัน		

ตัวอย่าง ตัน อยู่ช่วง

ค่าทุกน้ำหนัก

หน่วย $_2$ ต่อ กม แถวสีเขียว ค่าจาก

น้ำหนัก	ค่า
ตัน	
ตัน	
ตัน	
ตัน	
ตัน	
ตัน	
ตัน	
ตัน	
ตัน	

น้ำหนัก	ค่า
ตัน	
ตัน	
ตัน	
ตัน	
ตัน	
ตัน	
ตัน	
ตัน	

ความจุในกรณีนี้คือ ตัน ค่า ตันใช้เป็นจุดอ้างอิงของช่วง เท่านั้น
การประมาณแบบเส้นตรงเป็นสมมติฐาน แต่ค่าทั้งหมดอยู่ในช่วงแคบ ความคลาดเคลื่อนจึงมีจำกัด

ตัวอย่างการคำนวณ: รถคันที่ 1 (ลาน → 3 → 2 → 17 → ลาน)

ช่วง	ระยะ (กม.)	บนรถ	ค่าต่อ กม.	คาร์บอน (kg)
ลาน → 3	2.83	12 ตัน	0.7068	2.00
3 → 2	1.78	10 ตัน	0.6942	1.24
2 → 17	6.00	4 ตัน	0.6280	3.77
17 → ลาน	10.36	0 ตัน	0.5744	5.95
รวม	**20.98**			**12.96**

ที่มาของตัวเลขในตารางผลลัพธ์

รถที่ใช้	นับจากเส้นทางที่ได้ (จาก 8 คันที่มี)
ใช้ความจุ	83 ÷ (จำนวนคัน × 15) × 100
ระยะทางรวม	รวมระยะทุกช่วงของทุกคัน
คาร์บอน	ทำแบบตารางนี้ครบทุกคัน → Solver 156.4 · FIFO 193.1 kg
ร้อยละ	(FIFO − Solver) ÷ FIFO × 100

ผลเปรียบเทียบ FIFO กับ Solver

ตัวชี้วัด	FIFO	Solver	เปลี่ยนแปลง	ร้อยละ
รถที่ใช้	7 จาก 8 คัน	6 จาก 8 คัน	−1 คัน	−14.3%
ใช้ความจุตอนออกจากลาน	79.0%	92.2%	—	+13.2 จุด
ระยะทางรวม	302.9 กม.	243.7 กม.	−59.2 กม.	−19.5%
คาร์บอน	193.1 $kgCO_2e$/วัน	156.4 $kgCO_2e$/วัน	−36.7 $kgCO_2e$/วัน	−19.0%

ส่งครบ 20 จุด รวม 83 ตันเท่าเดิม



ผลจำลอง / ค่าการปล่อยจาก TGO / Thai National LCI Database รถบรรทุก 10 ล้อ

6 คันคือจำนวนขั้นต่ำทางทฤษฎี



จำนวนรถขั้นต่ำ = ⌈83 ÷ 15⌉ = 6 คัน

5 คัน ! ไม่พอ ขาด 8 ตัน



6 คัน ✓ เพียงพอ และเส้นทางที่ได้บรรทุกได้จริง



จำนวนรถ	ความจุรวม	เพียงพอต่อ Demand 83 ตันหรือไม่
5 คัน	75 ตัน	ไม่พอ ขาด 8 ตัน
6 คัน	**90 ตัน**	**เพียงพอ และเส้นทางที่ได้บรรทุกได้จริง**

» ภายใต้แบบจำลองหนึ่งเส้นทางต่อรถหนึ่งคันในรอบนี้ **จึงลดจำนวนรถต่ำกว่า 6 คันไม่ได้**

เส้นทางใหม่แบ่งเป็น 6 กลุ่ม

6 เส้นทาง ครอบคลุม 20 จุด และสินค้า 83 ตัน



ทุกเส้นทาง เริ่มและจบที่	น้ำหนักแต่ละเส้นทาง ไม่เกิน	Solver มีรถให้ใช้	ระยะทางรวม ตามตารางผลหลัก
Depot 1	15 ตัน	8 คัน และใช้จริง 6 คัน	243.7 กม.

เส้นทาง	ลำดับจุด (เริ่ม → จบ)	น้ำหนัก (ตัน)
V1	ลาน → 3 → 2 → 17 → ลาน	12
V2	ลาน → 5 → 7 → 4 → ลาน	15
V3	ลาน → 8 → 9 → 11 → ลาน	13
V4	ลาน → 10 → 12 → 13 → 14 → ลาน	15
V5	ลาน → 20 → 21 → 18 → 19 → ลาน	14
V6	ลาน → 16 → 6 → 15 → ลาน	14

เส้นทางรถคันที่ 1–3

รถ	ลำดับจุดส่ง	จุดส่ง	น้ำหนัก	ระยะทางที่รายงาน
V1	ลาน → 3 → 2 → 17 → ลาน	3	12 ตัน	21 กม.
V2	ลาน → 5 → 7 → 4 → ลาน	3	15 ตัน	15 กม.
V3	ลาน → 8 → 9 → 11 → ลาน	3	13 ตัน	35 กม.



น้ำหนักบรรทุก 12 / 15 ตัน



น้ำหนักบรรทุก 15 / 15 ตัน



น้ำหนักบรรทุก 13 / 15 ตัน

หมายเหตุ: ระยะรายคันแสดงตามต้นฉบับเป็นจำนวนเต็ม

เส้นทางรถคันที่ 4–6

รถ	ลำดับจุดส่ง	จุดส่ง	น้ำหนัก	ระยะทางที่รายงาน
V4  15/15 ตัน	ลาน → 10 → 12 → 13 → 14 → ลาน อ่อนนุช / พระโขนง / สุขุมวิท 101 / สำโรง	4 จุดส่ง	15 ตัน	49 กม.
V5  14/15 ตัน	ลาน → 20 → 21 → 18 → 19 → ลาน รามคำแหง / บวมินทร์ / คลองสามวา / หนองจอก	4 จุดส่ง	14 ตัน	62 กม.
V6  14/15 ตัน	ลาน → 16 → 6 → 15 → ลาน บางบ่อ / บางพลี / เทพารักษ์	3 จุดส่ง	14 ตัน	61 กม.
Σ รวม V1–V6		**20** จุด	**83** ตัน	**243.7** กม. ตามตารางผลหลัก



หมายเหตุ ปัดระยะแต่ละช่วงเป็นจำนวนเต็มตามมาตรฐาน ระยะรายคันจึงรวมได้ กม ส่วน กม เป็นระยะไม่ปัด ใช้เปรียบเทียบกับ และคำนวณคาร์บอน

คำตอบผ่านเงื่อนไขของโจทย์



Feasible

เอกสารรายงานว่าผ่านการตรวจสอบความถูกต้องครบ 10 ข้อ

เงื่อนไขสำคัญที่ระบุไว้ ได้แก่

1. ให้บริการครบทุกจุด
2. ไม่มีจุดใดได้รับบริการซ้ำ
3. ไม่มีรถคันใดบรรทุกเกิน 15 ตัน
4. ใช้รถไม่เกิน 8 คัน
5. ทุกเส้นทางเริ่มและจบที่ลานวัสดุ

6 คัน เป็นผลจากการจัดเส้นทาง โดยกำหนดรถพร้อมใช้ไว้ **8 คัน**

กรุณาเลือกไฟล์ .vrp จากเครื่องของคุณ

Choose Files CPAC-AGG-LKB-20.vrp

CPAC-AGG-LKB-20.vrp(n/a) - 726 bytes, last modified: 9/27/2026 - 100% done

Saving CPAC-AGG-LKB-20.vrp to CPAC-AGG-LKB-20.vrp

```
============================================================
  สรุปข้อมูลที่อ่านได้จากไฟล์
============================================================
  ชื่อไฟล์           : CPAC-AGG-LKB-20.vrp
  Depot              : Node 1
  จำนวน Customer     : 20
  Capacity           : 15
  max_vehicles       : 8  (จาก: COMMENT ในไฟล์)
  min vehicles need  : 6
  Demand รวม         : 83
  EDGE_WEIGHT_TYPE   : EUC_2D
```

```
============================================================
  Trace: 3 การเลือกแรกของ NN (รถคันที่ 1)
============================================================
 Step  Current Node  Selected Node  EUC_2D  Selected Demand  Load After Selection
    1             1             3       3                2                     2
    2             3             2       2                6                     8
    3             2             4       4                3                    11

============================================================
  Validation
============================================================
  ✅ PASS  1. Nodes exist in data: ทุก Node มีอยู่จริง
  ✅ PASS  2. All customers served: ครบทุกคน
  ✅ PASS  3. No duplicate customers: ไม่มีซ้ำ
  ✅ PASS  4. Capacity: ทุกรถไม่เกิน Capacity
  ✅ PASS  4b. Load match: Demand ตรงกันทุกคัน
  ✅ PASS  5. Vehicle count: 6 ≤ 8
  ✅ PASS  6. Start at depot: ทุก Route เริ่มที่ Depot
  ✅ PASS  7. End at depot: ทุก Route จบที่ Depot
  ✅ PASS  8. No depot in middle: ไม่มี Depot แทรกกลาง
  ✅ PASS  9-10. Distance match: ตรงกันทุกคัน (Total = 243)

------------------------------------------------------------
  OVERALL VALIDATION: PASS ✅
```

```
============================================================
  กำลังหา Route ที่ดีที่สุด...
============================================================
  NN + Local Search             : 257
  Multi-start NN + LS (best)    : 243
  Clarke-Wright + Local Search  : 247
  Sweep 36 มุม + LS (best)      : 243
  Exhaustive partition: ข้ามไป (รองรับรถ 2 คัน หรือ 3 คันที่ไม่เกิน 18 จุด · ไฟล์นี้มีรถ 8 คัน 20 จุด)
  Perturbation        : กำลังทำ (สูงสุด 50 รอบ)...
  Perturbation found  : 0 improvements

  ✅ วิธีที่ดีที่สุด: NN seed=4 + Local Search
  ✅ Total Distance : 243
  ⏱ เวลาที่ใช้      : 1.2 วินาที

============================================================
  ผลลัพธ์ที่ดีที่สุด
============================================================
 Vehicle                          Route  Customers  Demand  Cap Left  Distance
       1          1 -> 3 -> 2 -> 17 -> 1          3      12         3        21
       2           1 -> 5 -> 7 -> 4 -> 1          3      15         0        15
       3          1 -> 8 -> 9 -> 11 -> 1          3      13         2        35
       4 1 -> 10 -> 12 -> 13 -> 14 -> 1          4      15         0        49
       5 1 -> 20 -> 21 -> 18 -> 19 -> 1          4      14         1        62
       6         1 -> 16 -> 6 -> 15 -> 1          3      14         1        61

Total Distance = 243
```

หลายวิธีให้คำตอบใกล้เคียงกัน

วิธีทดสอบ	ผลที่รายงาน
	กม
	กม
	กม
มุม	กม
รอบ	ไม่พบการปรับปรุงเพิ่มเติม



เป็นคำตอบดีที่สุดเท่าที่ทดสอบตามรายงาน

ผลนี้ยังไม่ใช่หลักฐานยืนยัน Global Optimum ด้านระยะทาง หากไม่มีรายละเอียดขอบเขตการค้นหาและวิธีคำนวณที่ตรวจสอบได้

หมายเหตุ ทำงานเฉพาะโจทย์ที่ใช้รถ คัน
กรณีนี้มีรถ คันจึงไม่ได้ทำงาน

การกลับลานด้วยรถเปล่าลดลง

ตัวชี้วัดช่วงกลับรถเปล่า	FIFO	Solver	ลดลง
จำนวนเที่ยวกลับลาน	7	6	1 เที่ยว
ระยะทางกลับรวม	108.6 กม.	84.9 กม.	23.7 กม.
คาร์บอนช่วงกลับ	62.4 $kgCO_2e$	48.8 $kgCO_2e$	13.6 $kgCO_2e$

คาร์บอนช่วงกลับ = ระยะทางกลับ × 0.5744 $kgCO_2e$/กม.

คาร์บอนของช่วงรถเปล่าลดลง
21.8% จากเดิม

หมายเหตุ: 21.8% เป็นการลดของช่วงรถเปล่า ไม่ใช่การลดคาร์บอนรวม



37.1% ของคาร์บอนที่ลดได้มาจากช่วงรถเปล่า

ลดคาร์บอนรวม 36.7 $kgCO_2e$/วัน

แหล่งของการลด	ปริมาณ	สัดส่วนของยอดที่ลดได้
ช่วงกลับรถเปล่า	13.6 $kgCO_2e$/วัน	37.1%
ช่วงที่มีสินค้าบนรถ	23.1 $kgCO_2e$/วัน	62.9%

9.54 ตัน CO_2e/ปี/ลาน

คาร์บอนที่ลดได้ต่อปี

36.7 $kgCO_2e$/วัน × 260 วัน = 9,542 kg ≈ 9.54 ตัน CO_2e

260 วัน = 52 สัปดาห์ × 5 วันทำงาน · แปลงหน่วยด้วย ÷ 1,000 (kg → ตัน)
สมมติว่าทุกวันทำงานมีรูปแบบงานเหมือนกรณีตัวอย่าง



ส่งสินค้าเท่าเดิมด้วยการเดินทางน้อยลง

ปริมาณสินค้าที่ส่งยังคง
83 ตัน
ครบ 20 จุด

ระยะทางลดลง
19.5% ↓

คาร์บอนลดลง
19.0% ↓

รถที่ใช้ลดลง
1 คัน
จาก 7 เหลือ 6

ลดระยะทาง = ลดคาร์บอน ได้เกือบเต็มที่แล้ว

ข้อเท็จจริง

รถ 10 เล้อปล่อยคาร์บอนต่อ กม. ใกล้กันมาก
ไม่ว่าจะบรรทุกหรือไม่

รถเปล่า **0.57** · รถเต็ม **0.73** $kgCO_2e$/กม.

→ ระยะทางเป็นตัวกำหนดคาร์บอนหลัก ไม่ใช่น้ำหนัก

การทดสอบ

ทดสอบ: กลับทิศการวิ่งทุกเส้นทาง
ระยะทางเท่าเดิม แต่ลำดับส่งของเปลี่ยน น้ำหนักบนรถแต่ละช่วงจึงเปลี่ยน

ทิศตาม Solver	**156.41** $kgCO_2e$
เลือกทิศคาร์บอนต่ำสุด	**156.09** $kgCO_2e$
ลดเพิ่มได้เพียง	**0.2%**

ในกรณีนี้ เส้นทางที่สั้นที่สุดคือเส้นทางที่คาร์บอนต่ำเกือบที่สุดแล้ว



Solver ลดระยะทาง ไม่ได้ลดคาร์บอนโดยตรง
จึงทดสอบว่าเส้นทางที่ได้ห่างจากทางเลือกที่คาร์บอนต่ำกว่าแค่ไหน

ข้อจำกัด: ทดสอบเฉพาะการกลับทิศ
ยังไม่ได้จัดกลุ่มใหม่โดยใช้คาร์บอนเป็นเป้าหมาย

ข้อจำกัดด้านการปฏิบัติงานจริง

ข้อจำกัด	ผลต่อการนำไปใช้
 รถขนาดเดียว	แบบจำลองใช้รถ 15 ตันเหมือนกันทั้งหมด แต่ลานจริงอาจมีรถ 6 ล้อและ 10 ล้อ
 ระยะทางเส้นตรง	EUC_2D ไม่ใช่ระยะตามถนน ระยะและลำดับเส้นทางที่เหมาะสมอาจเปลี่ยน
 ไม่มีเงื่อนไขเวลา	ยังไม่รวมเวลารับของ เวลาขนถ่าย ชั่วโมงคนขับ และข้อจำกัดเวลาเดินรถในเมือง

ก่อนใช้งานจริงต้อง **เพิ่มข้อมูลและเงื่อนไขที่มีผลต่อการวิ่งรถ**

ข้อจำกัดของผลเปรียบเทียบและการคำนวณคาร์บอน

FIFO เป็นสมมติฐาน

ร้อยละที่ลดได้ยังไม่ใช่ผลวัดจาก
การปฏิบัติงานจริง

Solver ลดระยะทาง

ไม่ได้ค้นหาคำตอบที่ลดคาร์บอน
โดยตรง

ปัจจัยการปล่อย 4 ระดับ

ใช้การประมาณเส้นตรง
ระหว่างระดับบรรทุก

ไม่รวมสภาพการวิ่งทั้งหมด

ยังไม่รวมผลจราจร ความเร็ว
และการจอดรอที่ไซต์

หากเปลี่ยน เป้าหมายเป็นคาร์บอน

ลำดับส่งและการจัดกลุ่มอาจเปลี่ยน เพราะน้ำหนักบนรถแต่ละช่วง
มีผลต่อการปล่อย



ลำดับการพัฒนาต่อ

01 รองรับรถหลายขนาด

พัฒนาเป็น Heterogeneous Fleet VRP
เพื่อเลือกทั้งขนาดรถ จำนวนรถ และเส้นทาง
ให้ใกล้เคียงการทำงานจริง



02 ใช้ระยะทางถนนจริง

แทนระยะตรงด้วยข้อมูลเครือข่ายถนน
เพื่อสะท้อนระยะวิ่งที่ใช้งานได้จริง



03 เพิ่มเป้าหมายด้านคาร์บอน

เปรียบเทียบคำตอบที่ลดระยะทาง
กับคำตอบที่ลดการปล่อยคาร์บอน



CVRP ช่วยให้กองรถรองรับงานได้ดีขึ้น

ส่งครบ 20 จุด รวม 83 ตัน ด้วยรถ 6 คัน



ใช้รถลดลง 1 คันจาก FIFO
และมีรถว่างเพิ่มสำหรับโอกาสรับงานใหม่

ระยะทางลดลง 59.2 กม.
หรือ **19.5%**

คาร์บอนลดลง 36.7 $kgCO_2e$/วัน
หรือ **19.0%**

เทียบเป็นประมาณ 9.54 ตัน CO_2e/ปี
ที่ 260 วันทำงาน

37.1% ของคาร์บอนที่ลดได้
มาจากช่วงกลับรถเปล่า

ครบทุกจุด ภายใต้ข้อจำกัดของกรณีศึกษา

สิ่งที่เรียนรู้

รถ ล้อตอนเปล่าและตอนเต็มปล่อยคาร์บอนต่อ กม ใกล้กัน เทียบ
ระยะทางจึงเป็นตัวกำหนดหลัก ในกรณีนี้ลดระยะทางก็คือลดคาร์บอนไปพร้อมกัน

หมายเหตุ: ต้องทดสอบด้วยข้อมูลถนน เวลา และการปฏิบัติงานจริงก่อนขยายผล